Separable Box-Constrained Sum-Equality QP Layer — Demo (Figures & Interpretation)

This report explains the figures produced by the demo script in `demo_outputs_v2/` for a **separable convex QP** with a **single sum-equality constraint** and **box constraints**.

All image links are **relative** so the file should sit in the same directory as the `.png` outputs.

0. Problem and notation

We solve, for each row independently:

$$\min_{x\in\mathbb{R}^N} \sum_{i=1}^{N}\left(\gamma_i x_i^2 + \beta_i x_i\right)$$

$$\text{s.t.} \sum_{i=1}^{N} x_i = \xi, \qquad m_i \le x_i \le M_i, \qquad \gamma_i > 0.$$

Ignoring constants, this is a **weighted projection** of z to set $\mathcal{C}$:

$$\min_x \sum_{i=1}^{N} \gamma_i (x_i - z_i)^2, \qquad z_i = -\frac{\beta_i}{2\gamma_i}.$$

$$\mathcal{C} = \left\{ x \in \mathbb{R}^N : \sum_i x_i = \xi,\ m_i \le x_i \le M_i \right\}.$$

1. Closed-form structure and the role of the multiplier

1.1 KKT form (the identity the layer implements)

The solution has a scalar Lagrange multiplier λ (for the equality constraint) such that:

$$x_i^*(\lambda) = \text{clip}\left(z_i + \frac{\lambda}{2\gamma_i},\ [m_i, M_i] \right),$$

and λ is chosen so that:

$$\sum_{i=1}^{N} x_i^*(\lambda) = \xi.$$

Define the monotone function:

$$g(\lambda) = \sum_{i=1}^{N} \text{clip}\left(z_i + \frac{\lambda}{2\gamma_i},\ [m_i, M_i] \right).$$

Since each clipped term is monotone in λ, $g(\lambda)$ is monotone, so there is a unique solution whenever $\xi \in [\sum_i m_i, \sum_i M_i]$.

1.2 Practical computation of λ (why the "active set" isn't scary)

Although the Bayón et al. piecewise solution can be described using active sets, in implementation you can avoid explicit regime enumeration:

1. Find a bracket $[\lambda_{\min}, \lambda_{\max}]$ with $g(\lambda_{\min}) \le \xi \le g(\lambda_{\max})$.
2. Solve $g(\lambda) = \xi$ by **bisection**.

This is robust, deterministic, and needs only evaluations of $g(\lambda)$, which are vectorized.

Cost: $O(B \cdot N \cdot T)$ with batch B, dimension N, and bisection iterations T (often 30–80 is plenty).

2. How to read the printed stats (so the numbers feel "anchored")

A few anchors help interpret magnitudes:

- In **simplex mode** ($m = 0$, $M = 1$, $\xi = 1$), each row sums to 1, so the *average* entry should be near $1/N$.
  - Example: if $N = 100$, expected mean ≈ 0.01.
- **Softmax** is dense: all entries strictly positive, so `avg_L0_per_row ≈ N`.
- **QP projection** often has exact zeros, so `avg_L0_per_row` is typically small (sparse support).
- "Fast vs high-iter error" plots should be concentrated near 0 if bisection is converging well.

3. Use case A — Simplex-like activation (QP projection vs Softmax)

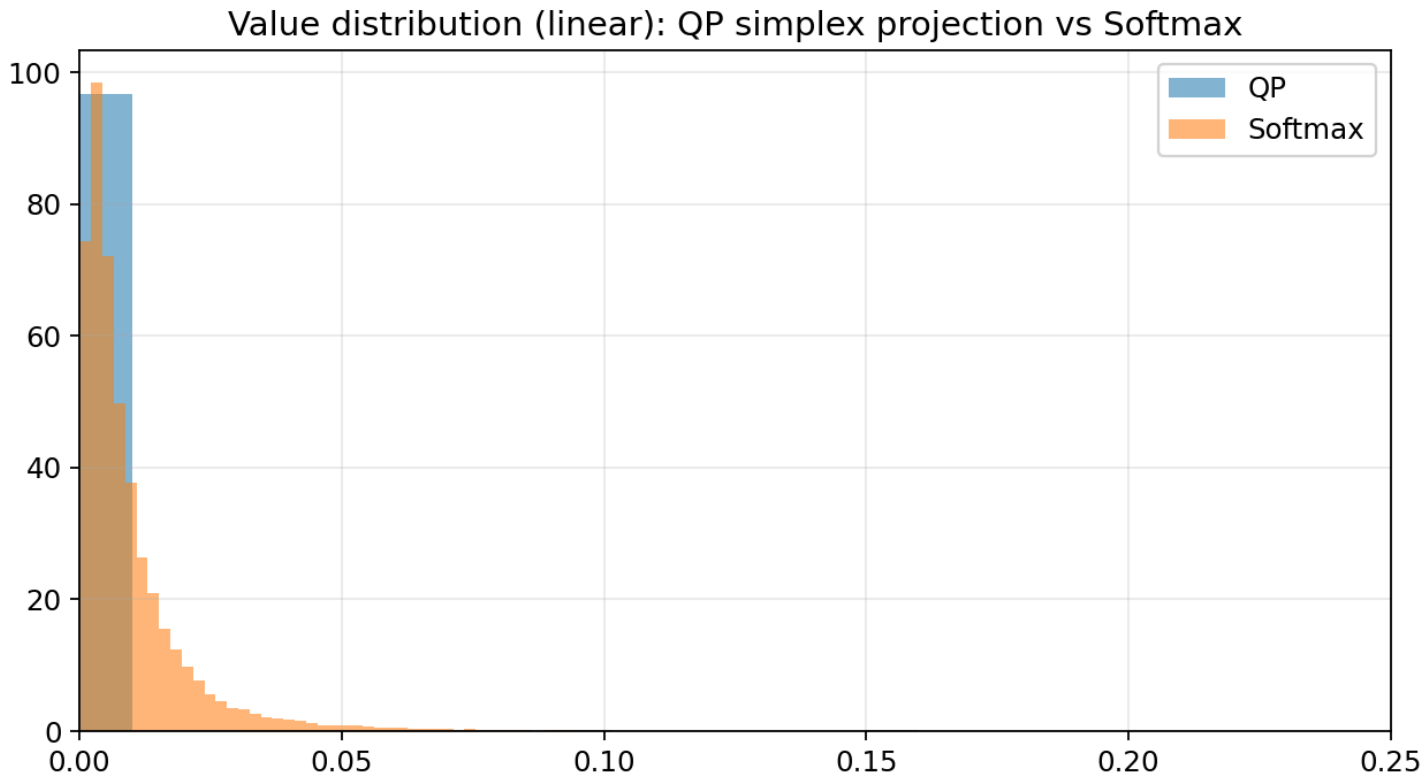
3.1 Setup

- $m_i = 0$, $M_i = 1$
- $\xi = 1$
- $\gamma_i = 1$

This maps logits z to a vector in $[0, 1]^N$ with sum exactly 1. It behaves like "probabilities" but can be **sparse**.

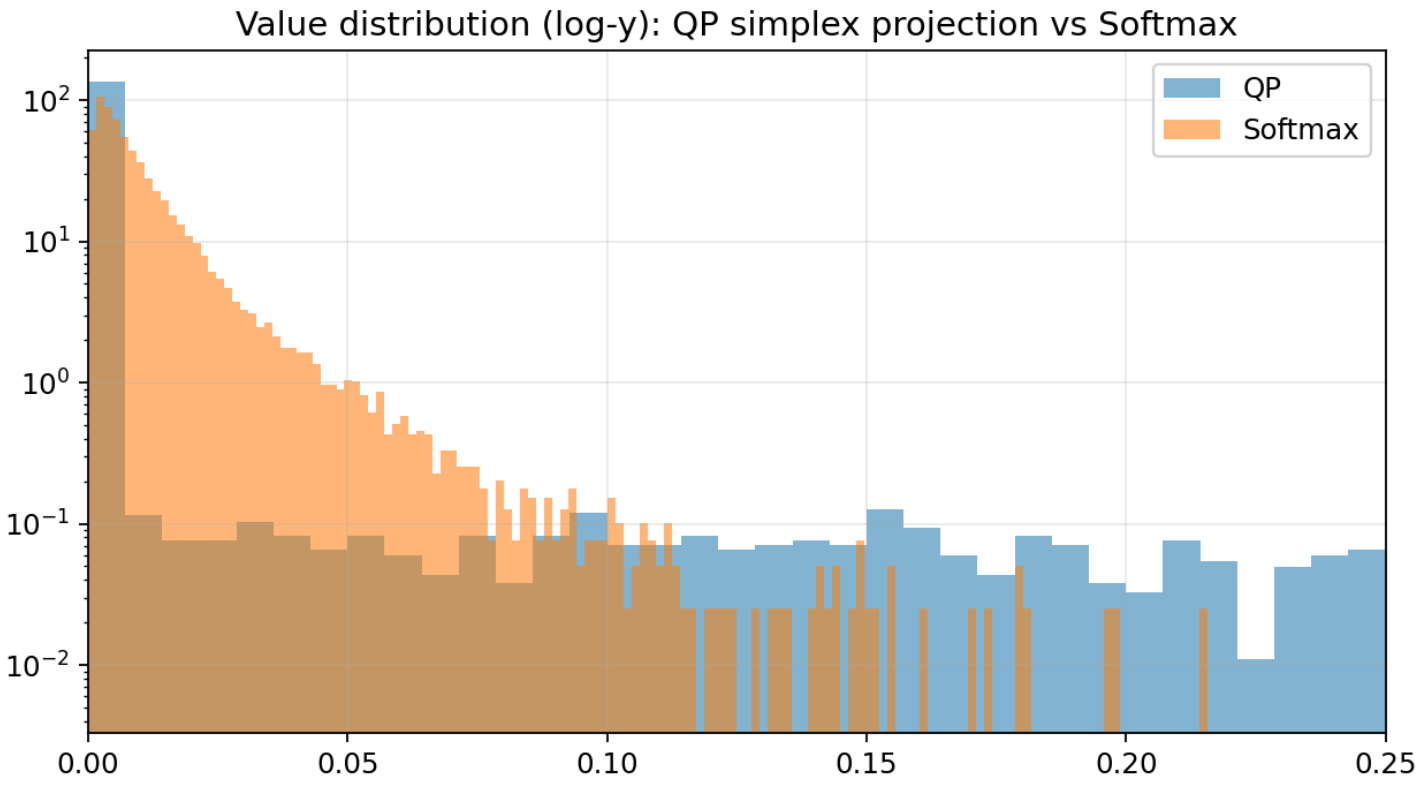
3.2 Distribution of values

Linear-scale histogram



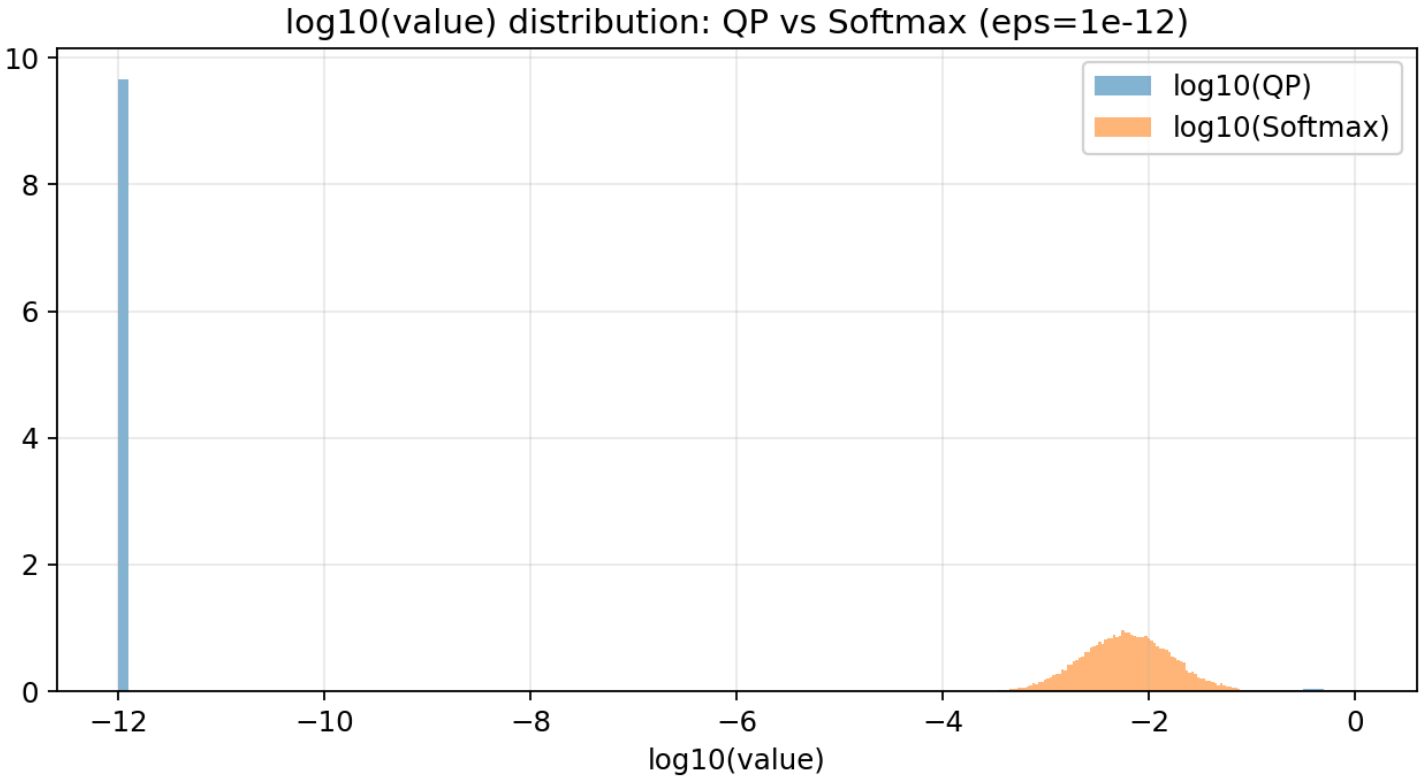
This emphasizes the nonzero region (often values in $[0, 0.2]$ when $N = 100$).

Log-y histogram



Log-y reveals the enormous mass at **exactly 0** for QP projection.

Log10(value) histogram

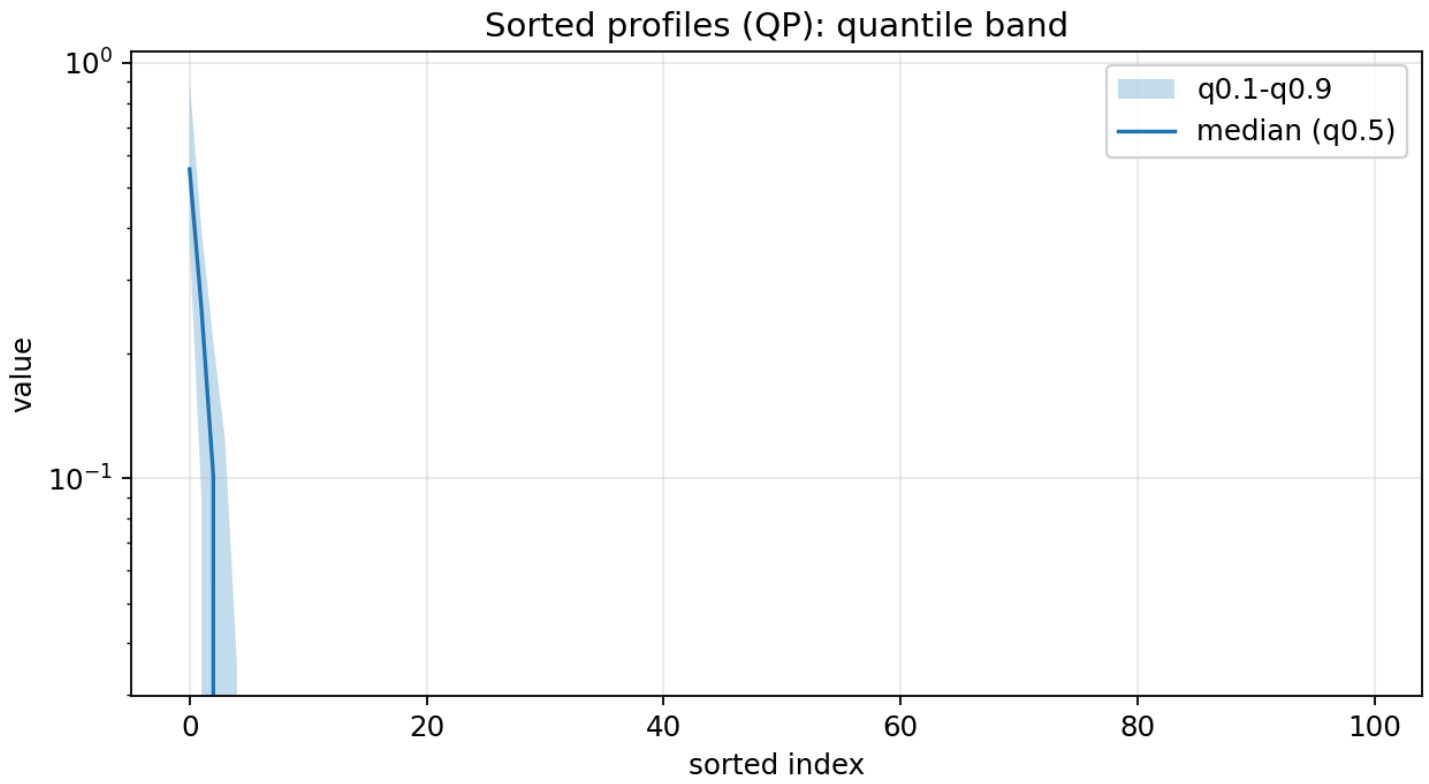


This compresses scale and tends to show a large spike near the plotting epsilon because zeros are replaced by a tiny value for log visualization.

3.3 Sorted profile quantile bands (shape + sparsity)

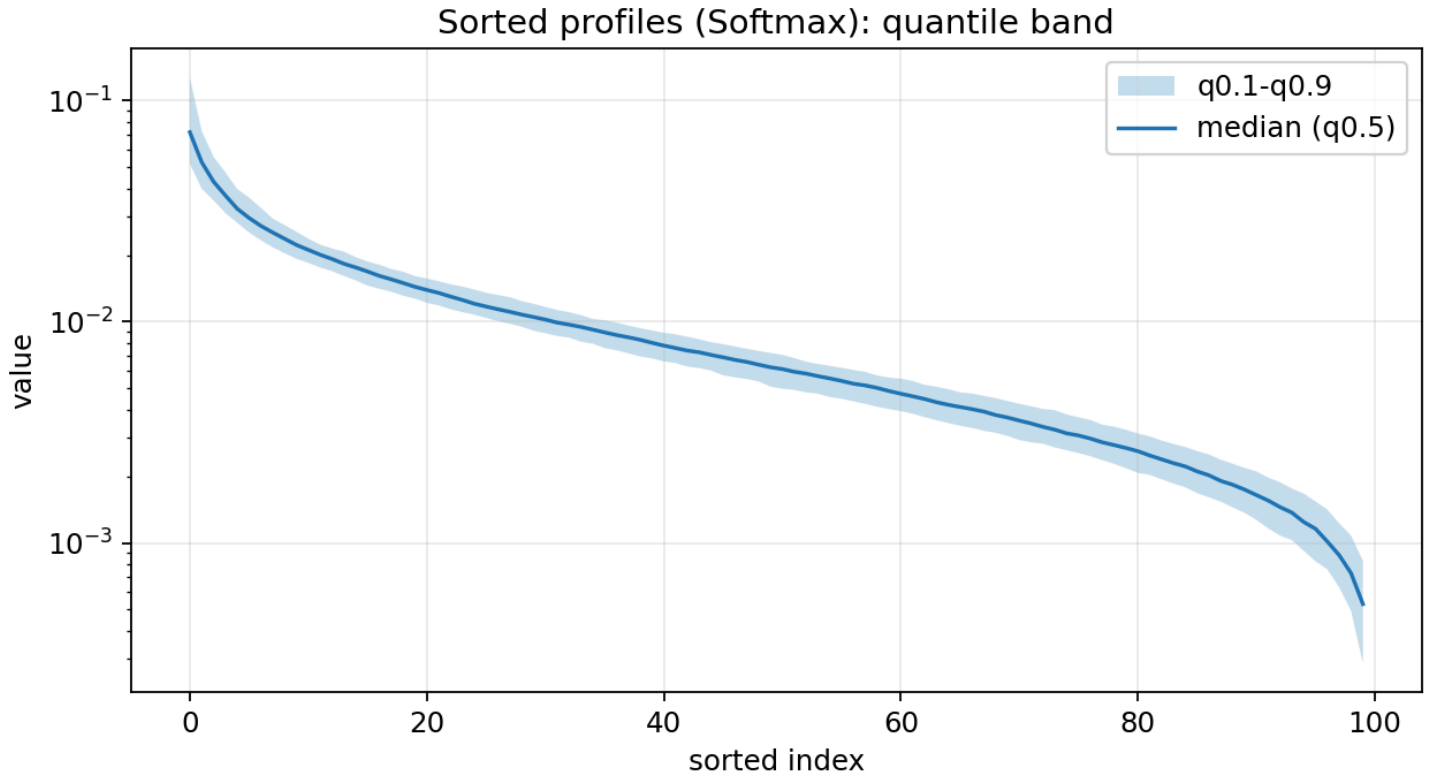
Instead of plotting many trajectories, we summarize across the batch by quantiles after sorting each row descending.

QP projection



Expect a sharp drop and then near-zeros: this is the hallmark of projection-induced sparsity.

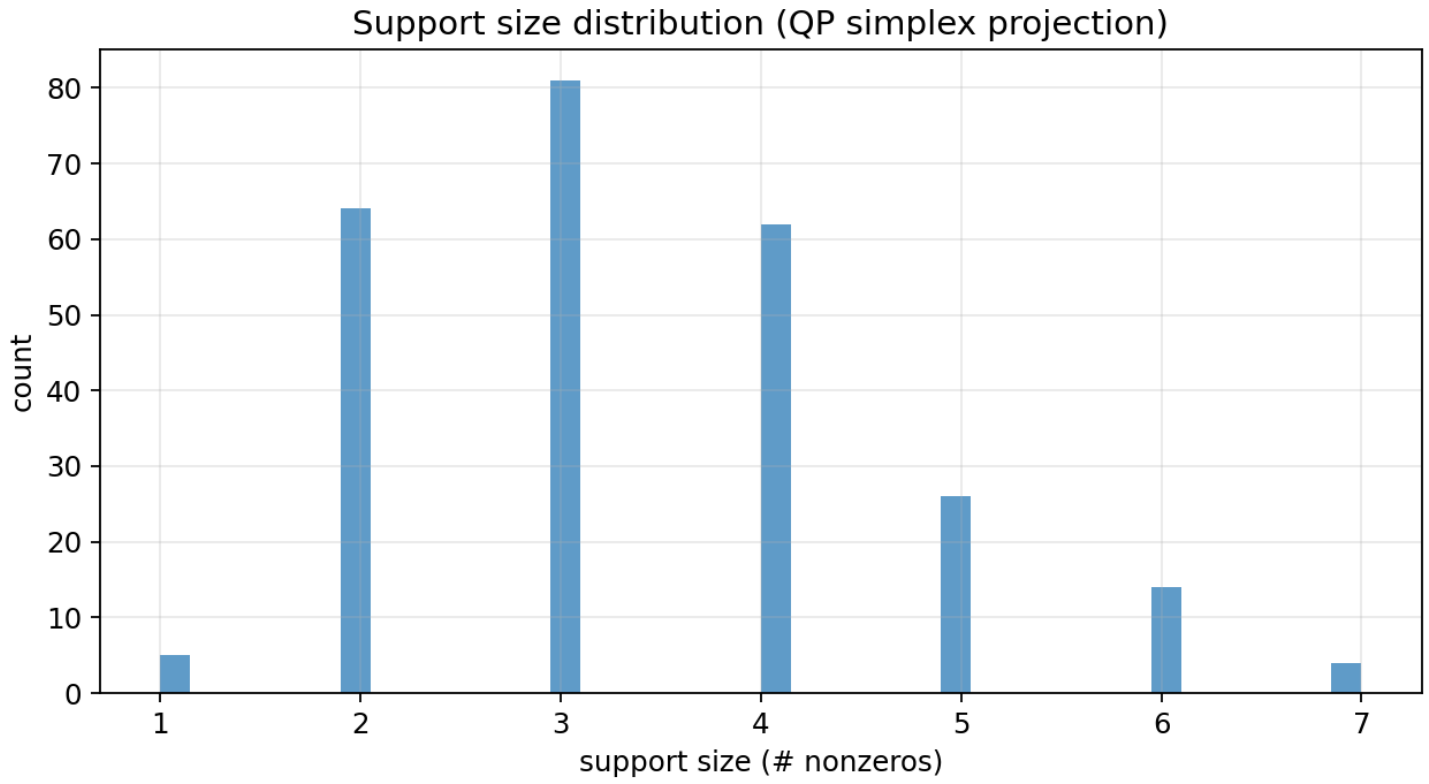
Softmax



Expect a smooth tail: softmax allocates small mass across *all* coordinates.

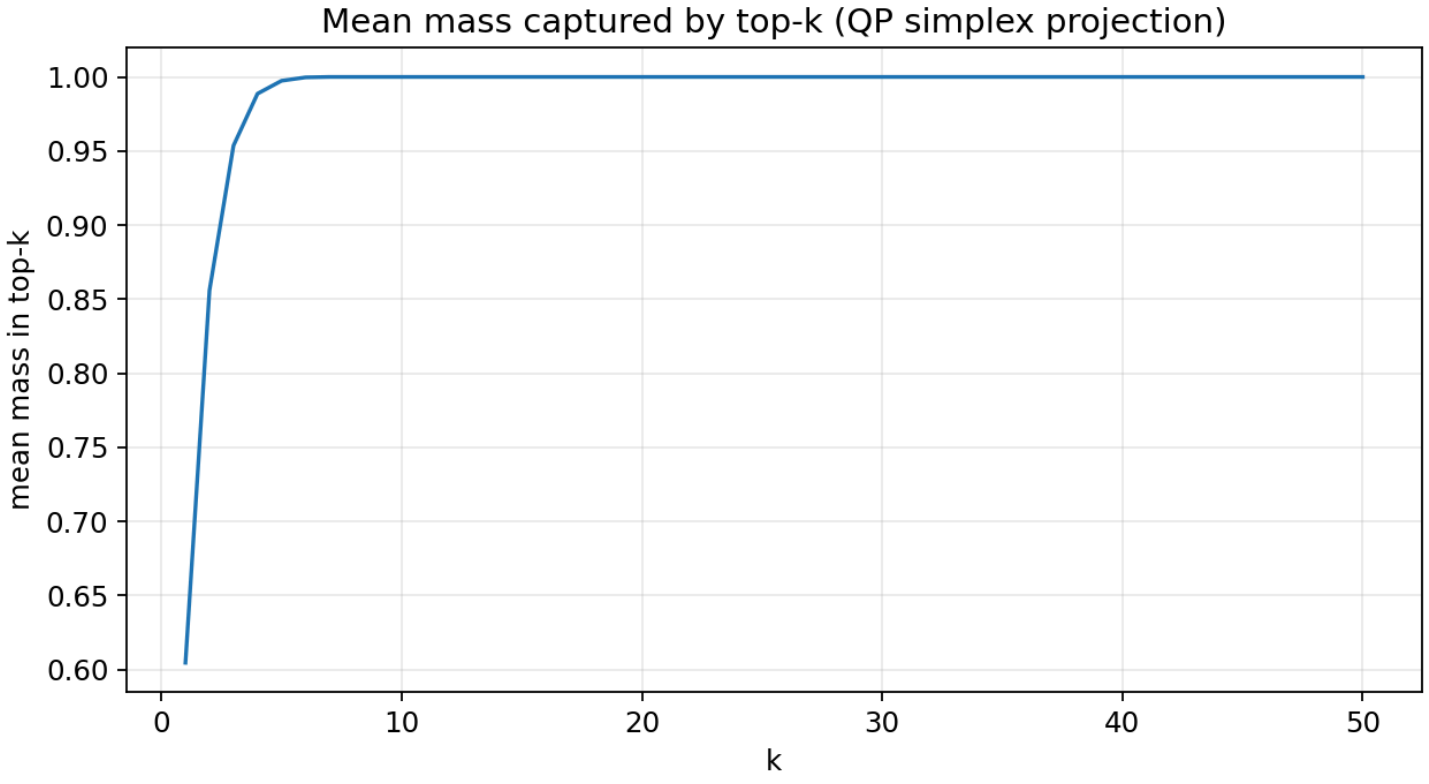
3.4 Sparsity summary and “how concentrated is the mass?”

Support size (number of nonzeros)



This is the most interpretable sparsity metric: how many coordinates are active per row.

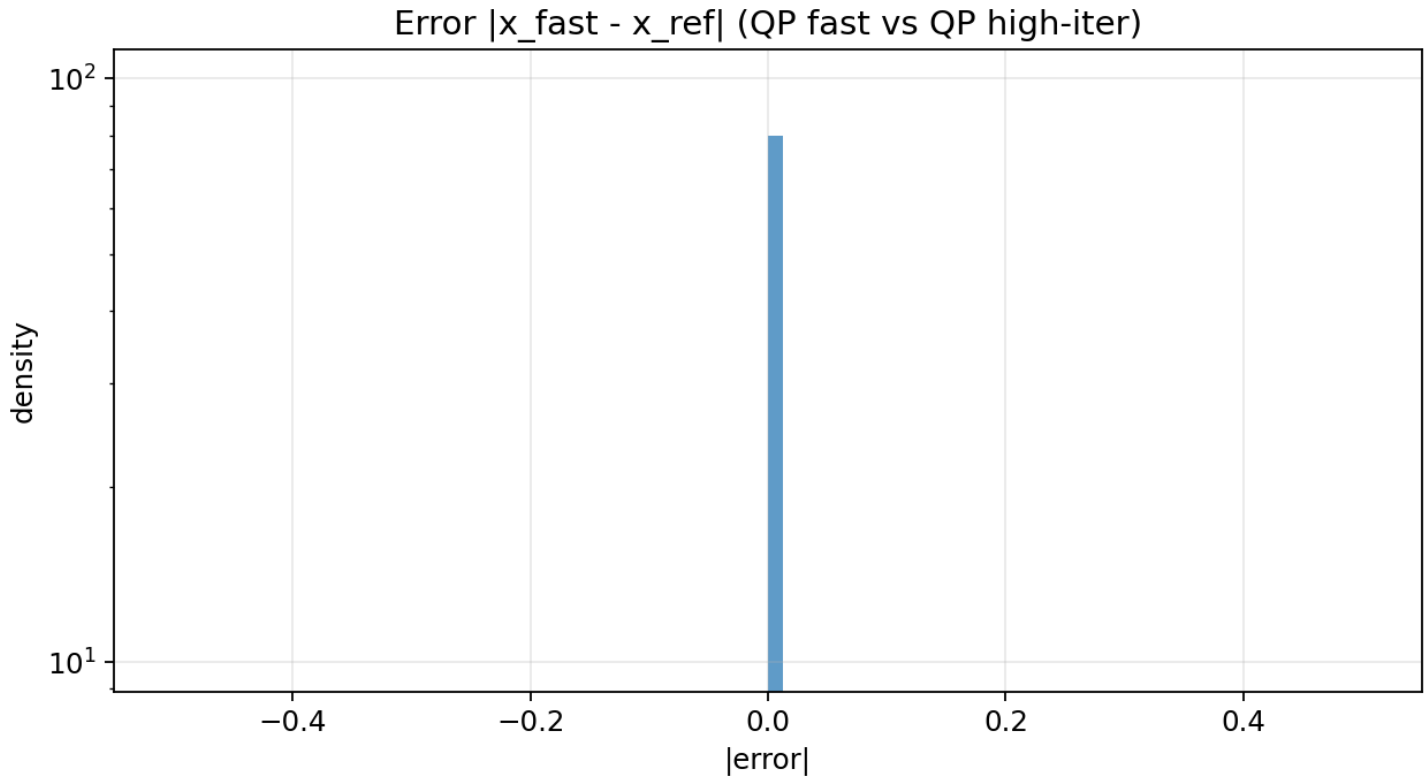
Top-k mass curve



This answers: "How many coordinates do I need to capture (say) 90% of the mass?"
If the curve rises quickly, the representation is highly concentrated.

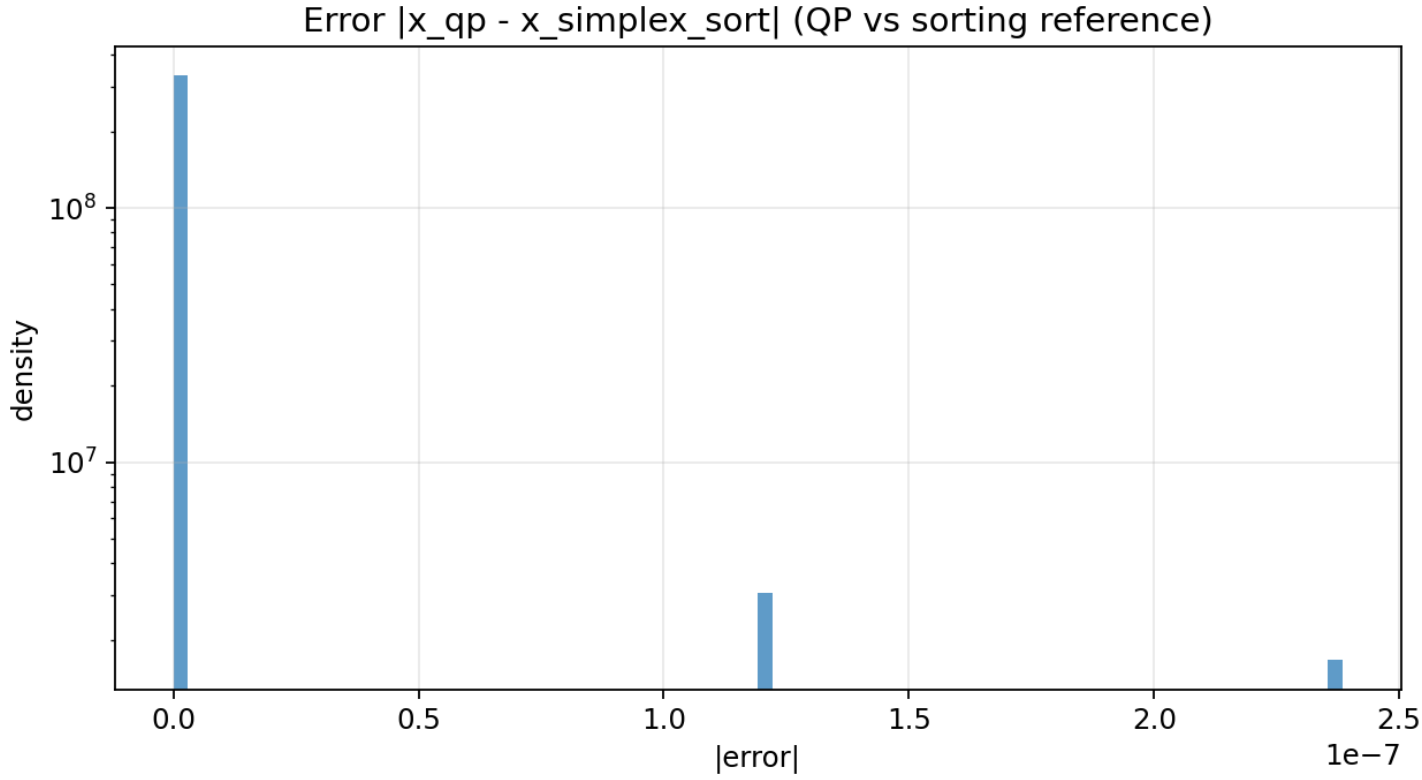
3.5 Numerical cross-checks

Fast bisection vs high-iter reference



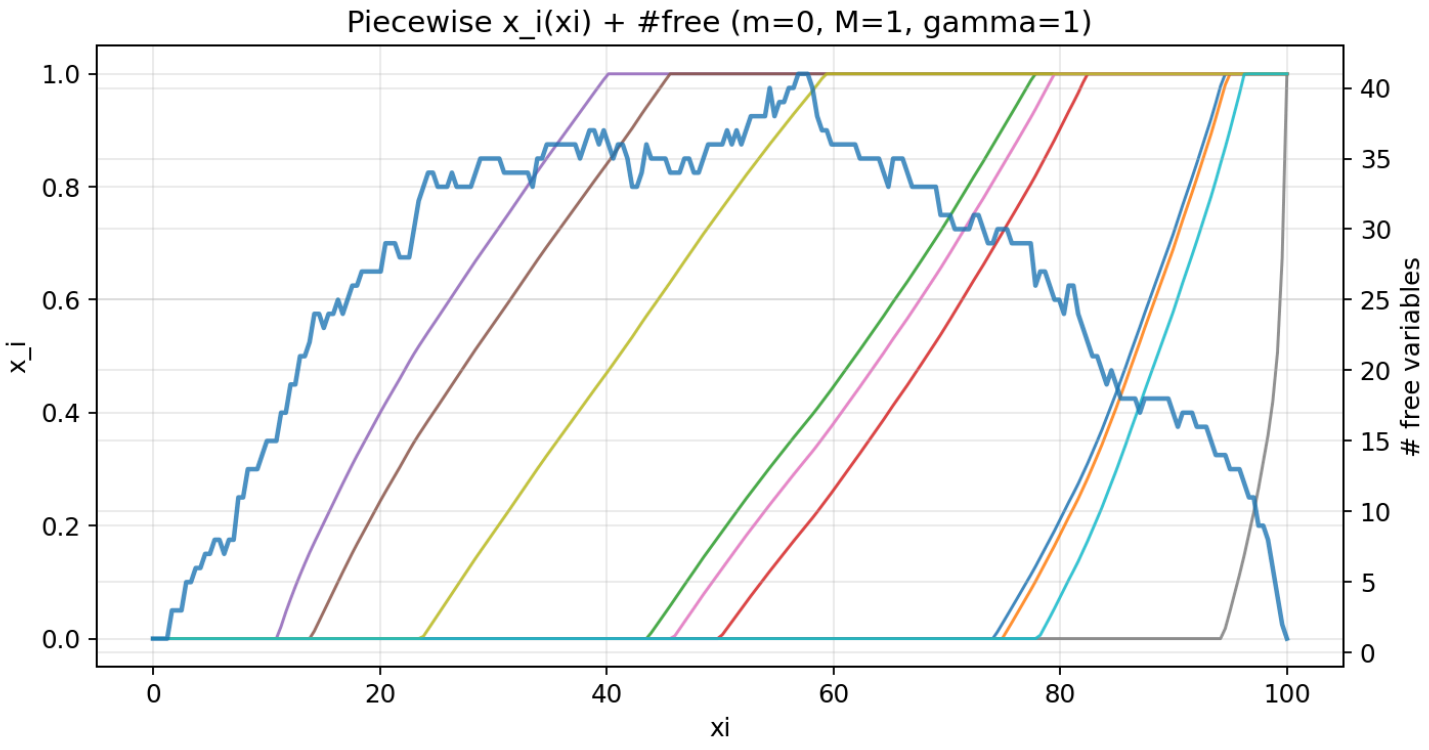
If this looks like a tight spike near 0, your chosen iteration count is adequate.

QP projection vs sorting-based simplex projection reference



In the common case where the upper bound is inactive, QP projection should match classic simplex projection closely.

3.6 Piecewise structure as ξ changes (active-set changes)



This plot sweeps ξ and shows how individual coordinates "turn on," saturate, or remain at 0.

- Between active-set changes, $x^*(\xi)$ is **affine** in ξ.
- When a coordinate hits 0 or 1, the active set changes and the slope pattern changes.

4. Use case B — k-hot / budgeted gating ($x \in [0,1]$, $\sum x = k$)

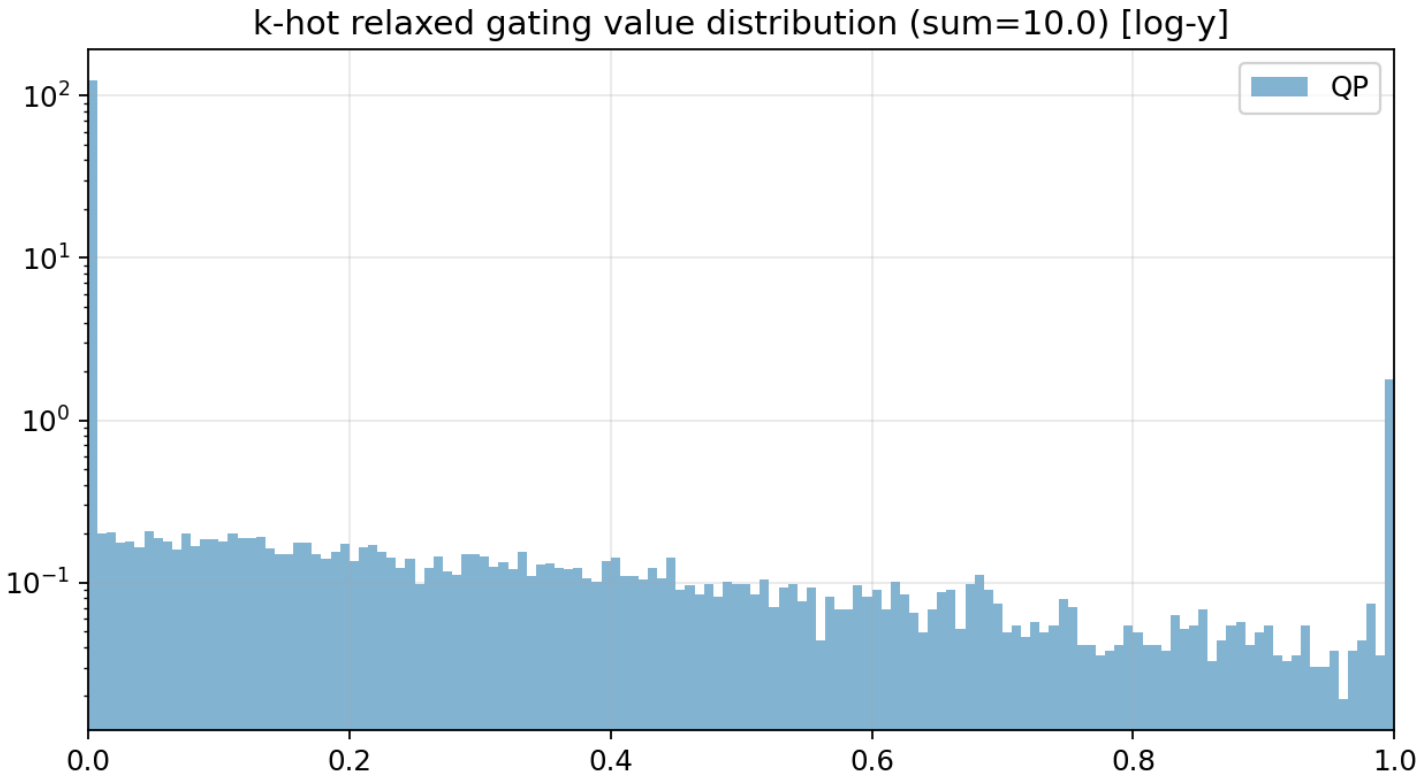
4.1 Setup

- $m_i = 0$, $M_i = 1$
- $\xi = k$ (demo uses $k = 10$)
- $\gamma_i = 1$

Interpretation: distribute a fixed "budget" k across N channels, with caps.

4.2 Distributions and profiles

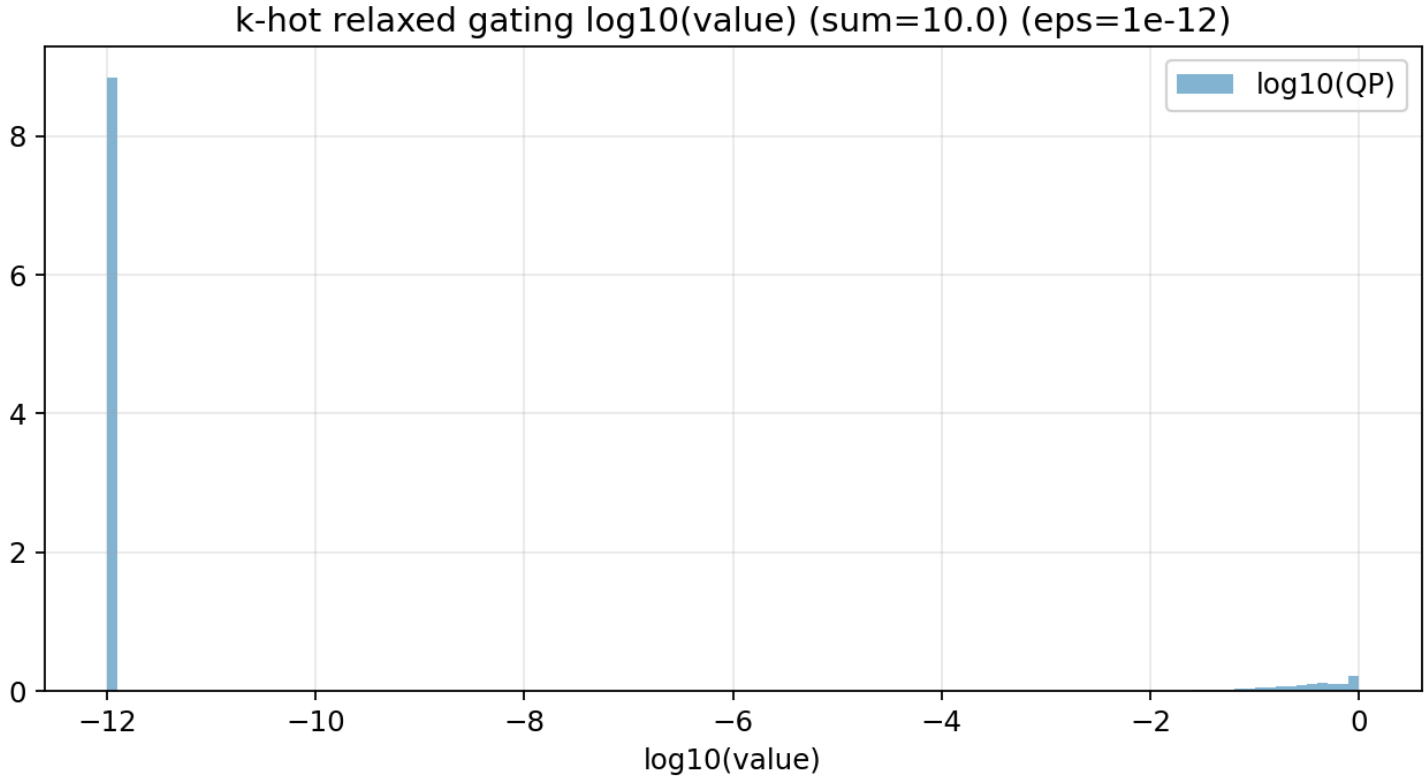
Value distribution (log-y)



Often shows:

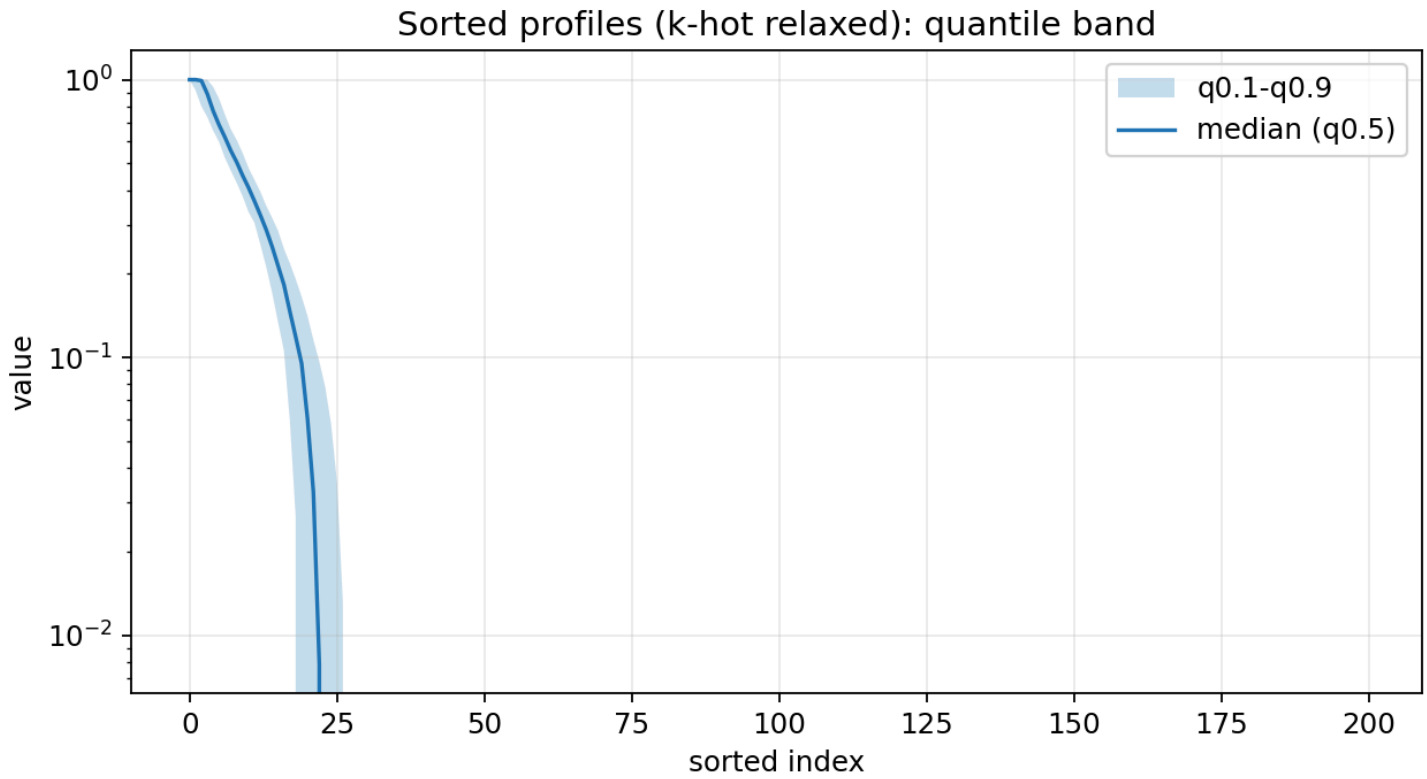
- many zeros,
- some fractional values,
- sometimes saturation at 1.

Log10(value) view



Like in simplex mode, zeros create a dominant spike at the epsilon floor.

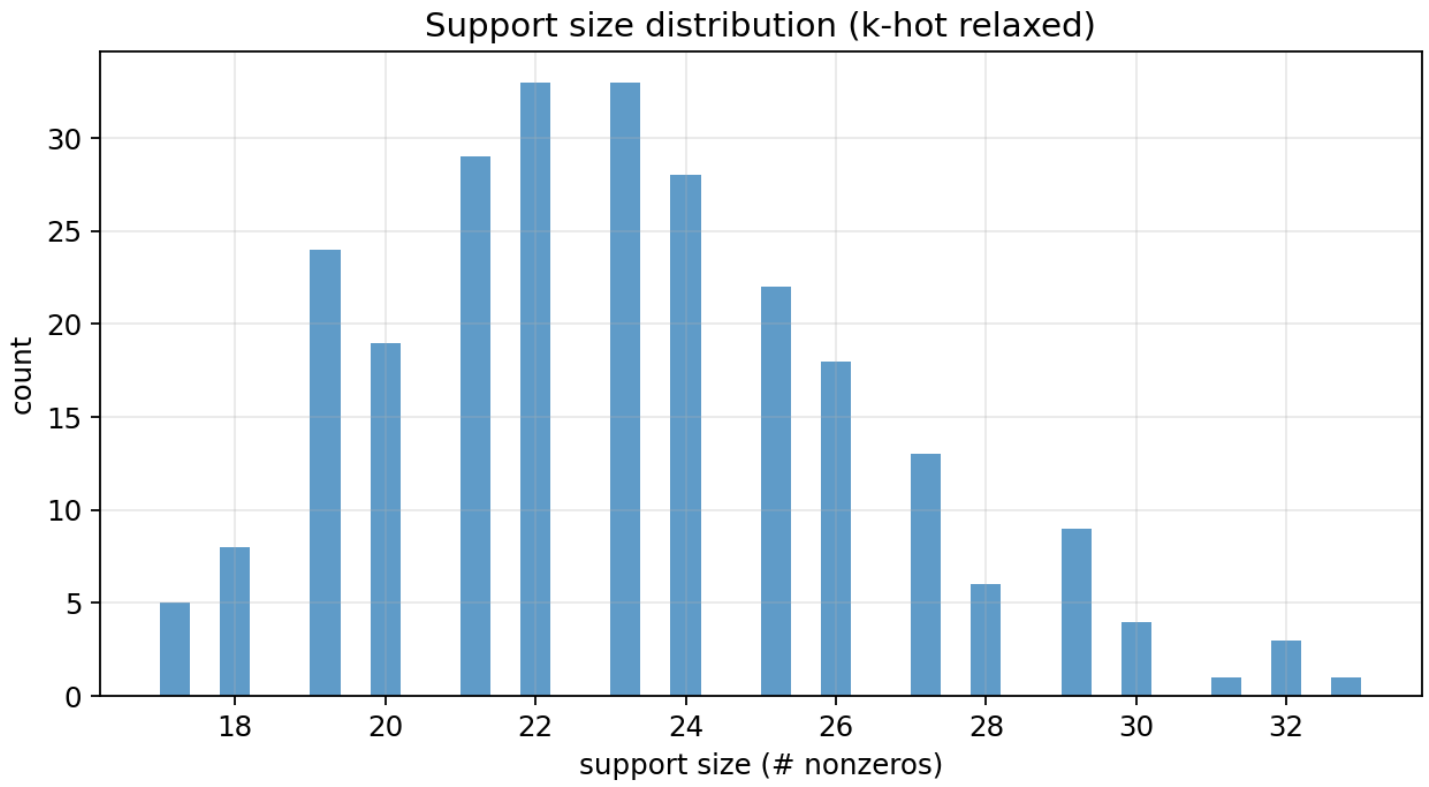
Sorted profile quantile band



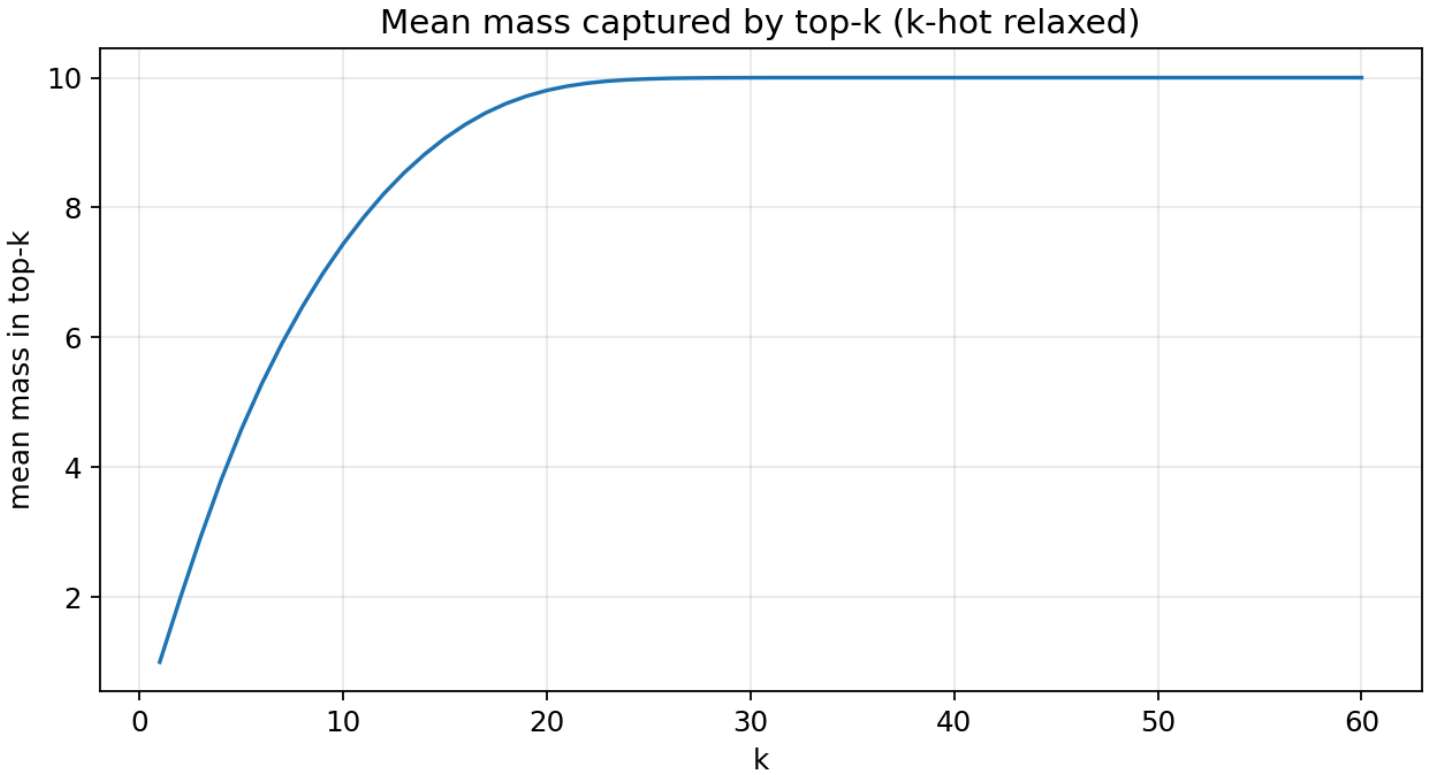
Compared to simplex ($\xi = 1$), the top of the profile is "fatter" because there is more total mass.

4.3 Sparsity and concentration

Support size

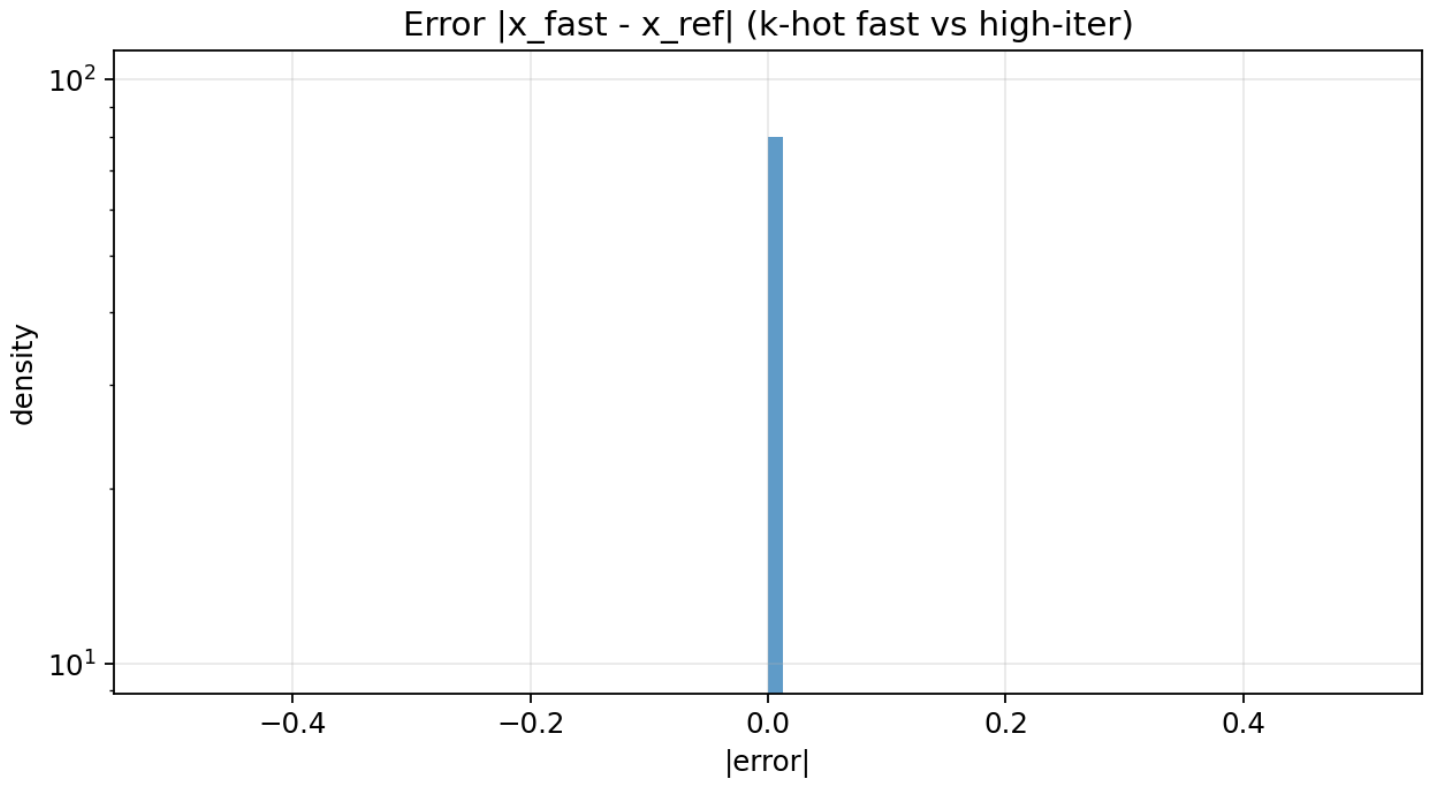


Top-k mass curve



This shows how quickly the budget concentrates in the largest few coordinates.

4.4 Numerical accuracy (fast vs high-iter)



As above: should be near-zero unless bisection iterations are too low.

5. Use case C — Adaptive ξ (learnable mass, always feasible)

5.1 Setup (concept)

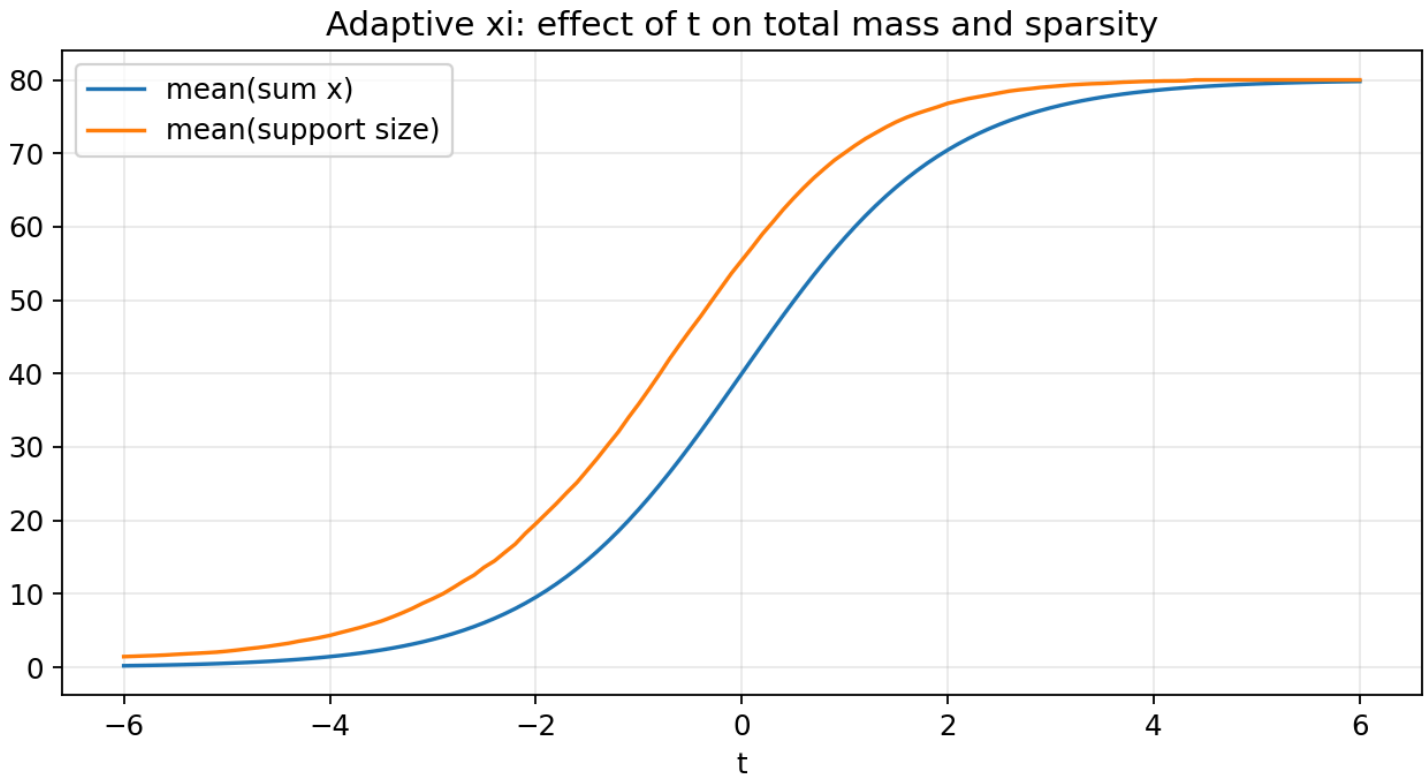
A common parameterization ensures feasibility automatically:

$$\xi(t) = \sum_i m_i + \sigma(t)\big(\sum_i M_i - \sum_i m_i\big),$$

so ξ always lies in the feasible interval.

With $m = 0, M = 1$, this becomes $\xi(t) \in [0, N]$.

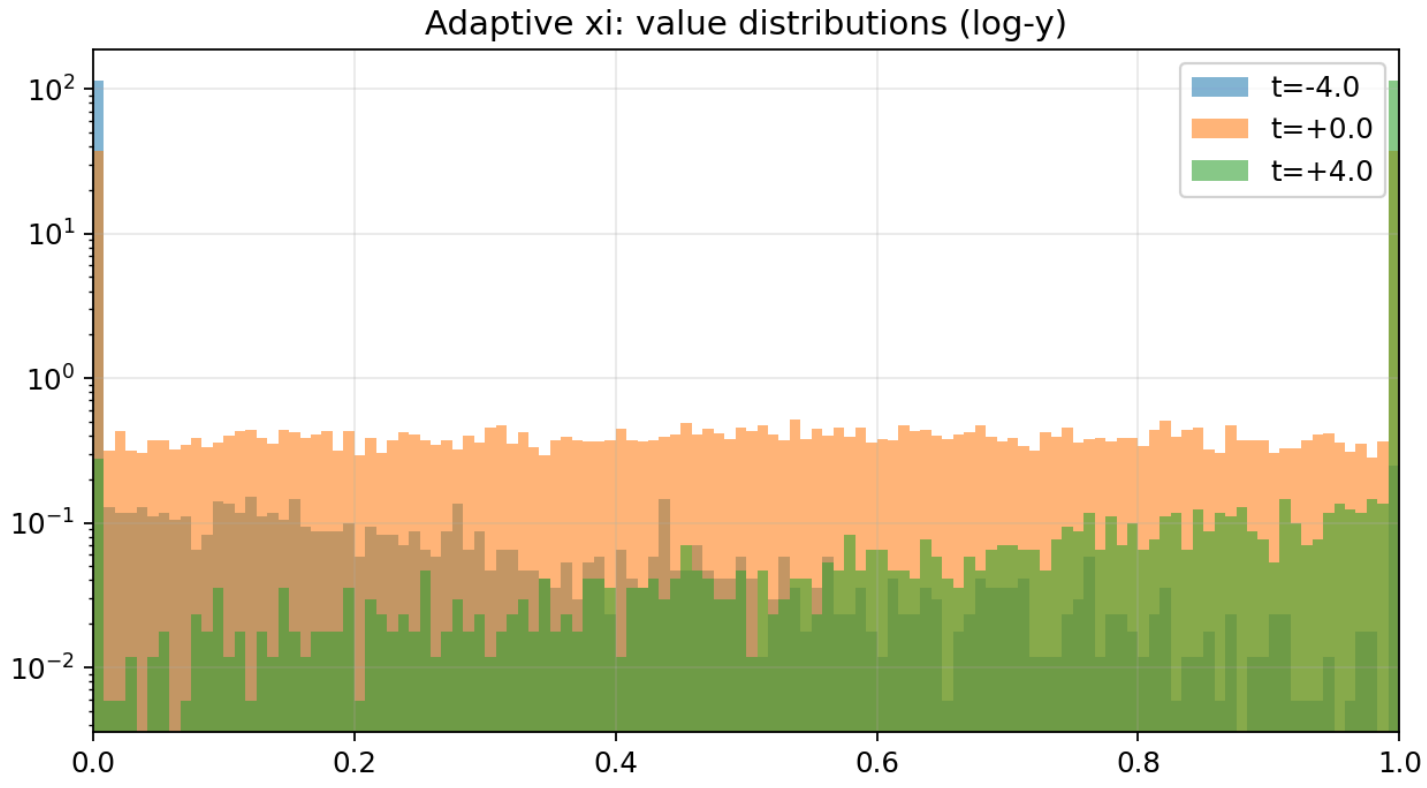
5.2 Figure: ξ sweep effect



This plot typically shows:

- as t increases, mean row-sum increases (more budget),
- sparsity often decreases (more coordinates become active).

Value distribution for an example setting



6. Use case D — Learnable bounds (m_i, M_i learned safely)

6.1 Why learn bounds?

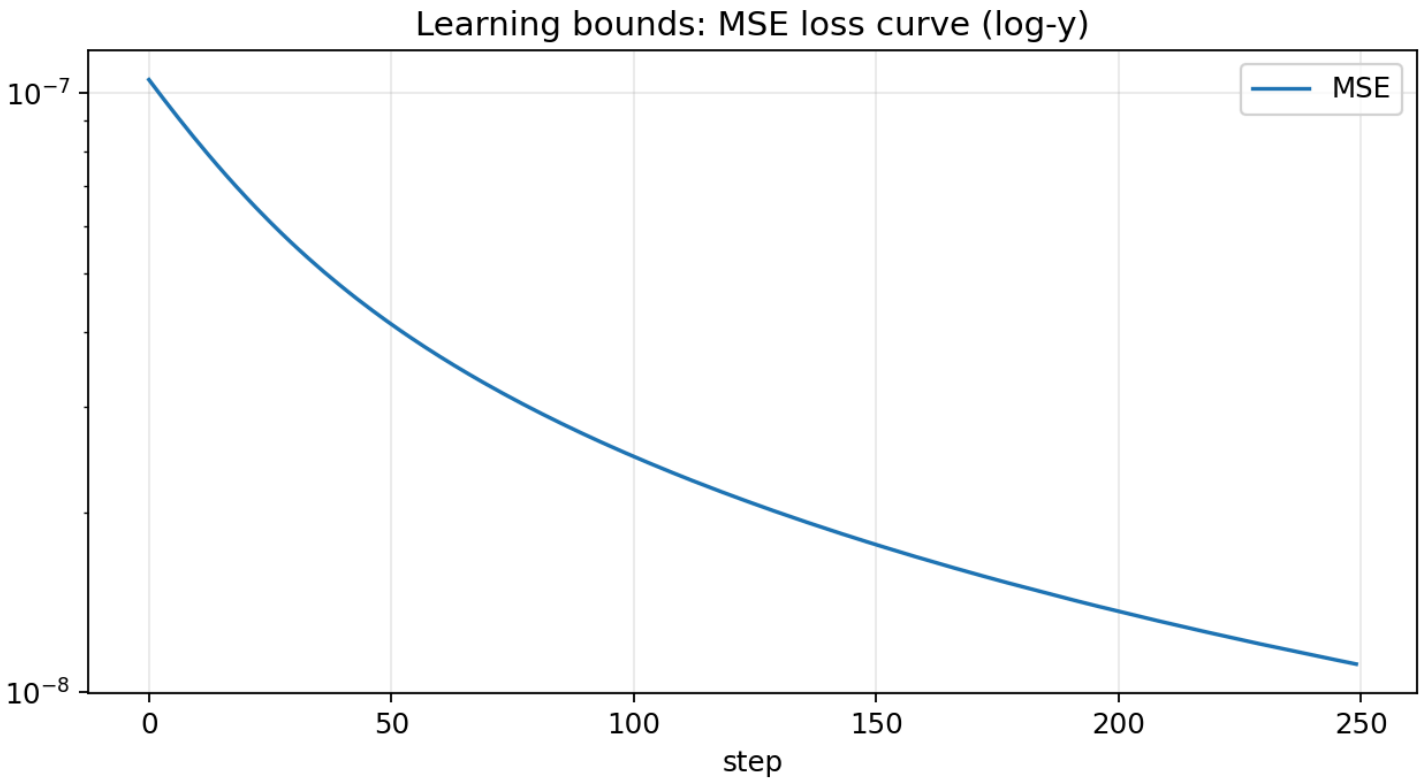
Learning bounds lets the model learn the feasible polytope itself:

- "disable" coordinates by shrinking $M_i - m_i$,
- enforce baseline allocations via m_i,
- cap capacity via M_i.

A safe construction ensures $m_i < M_i$ always (e.g., nested sigmoids), keeping optimization stable.

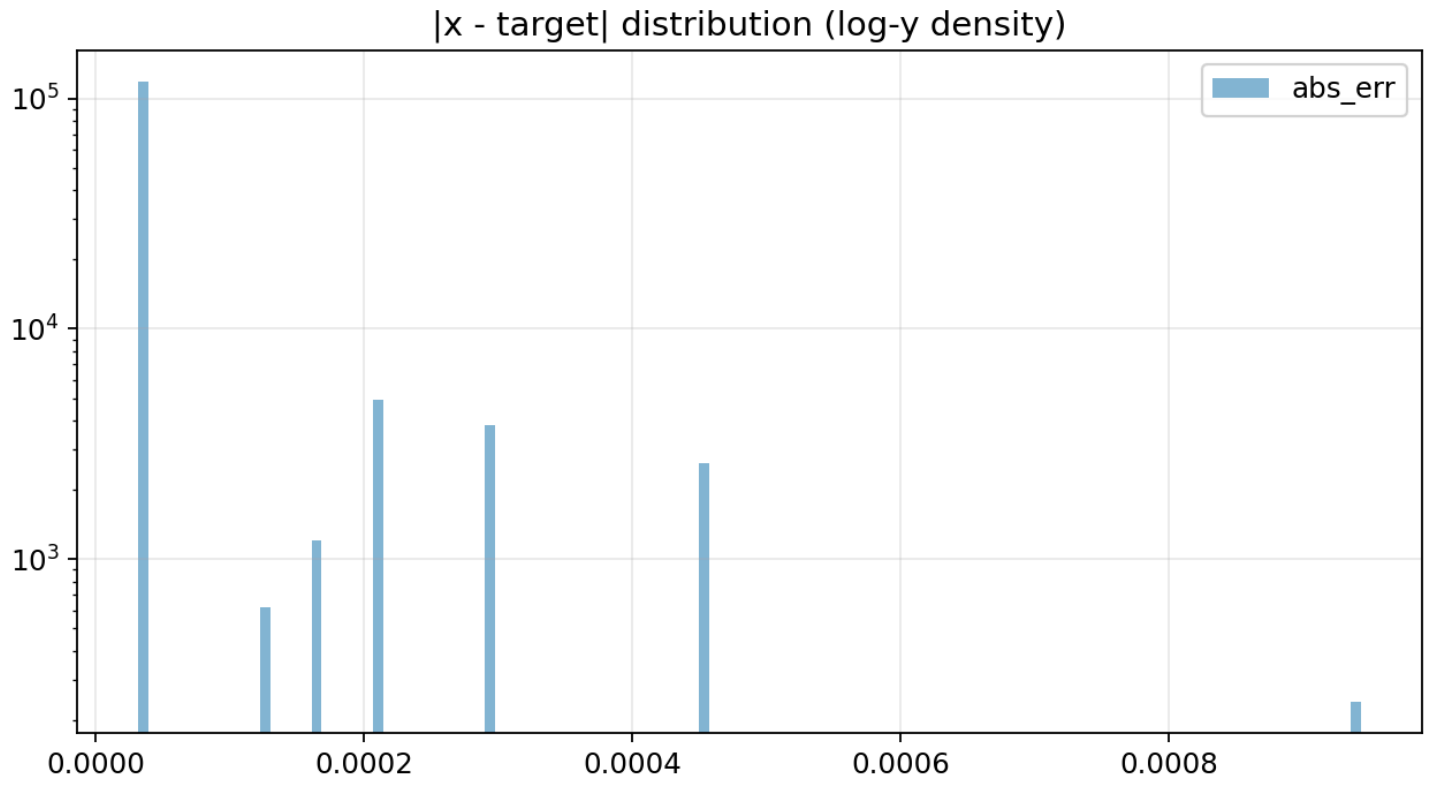
6.2 Training diagnostics

Loss curve (log-y)



A monotone decrease indicates gradients propagate through the layer.

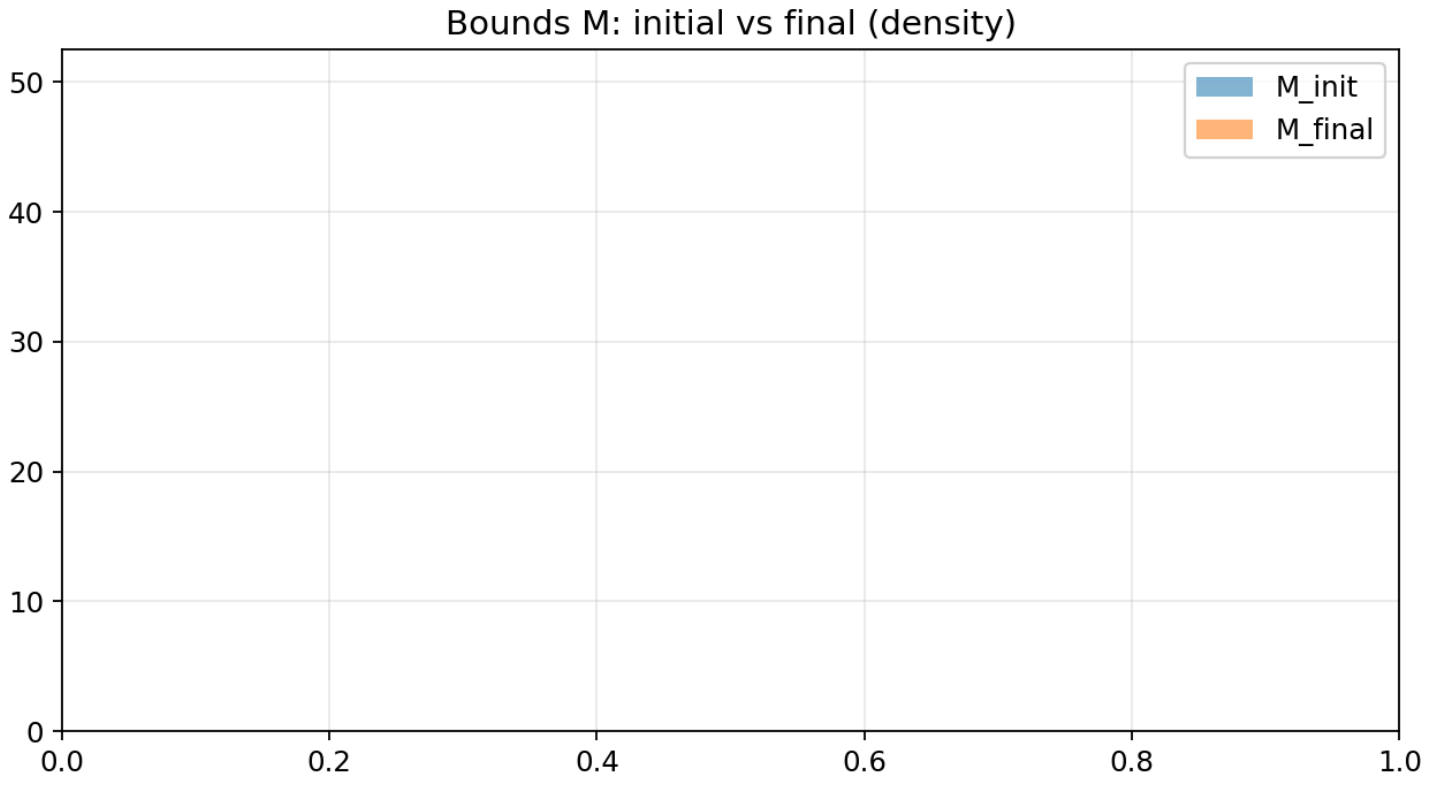
Absolute error distribution (log-y)



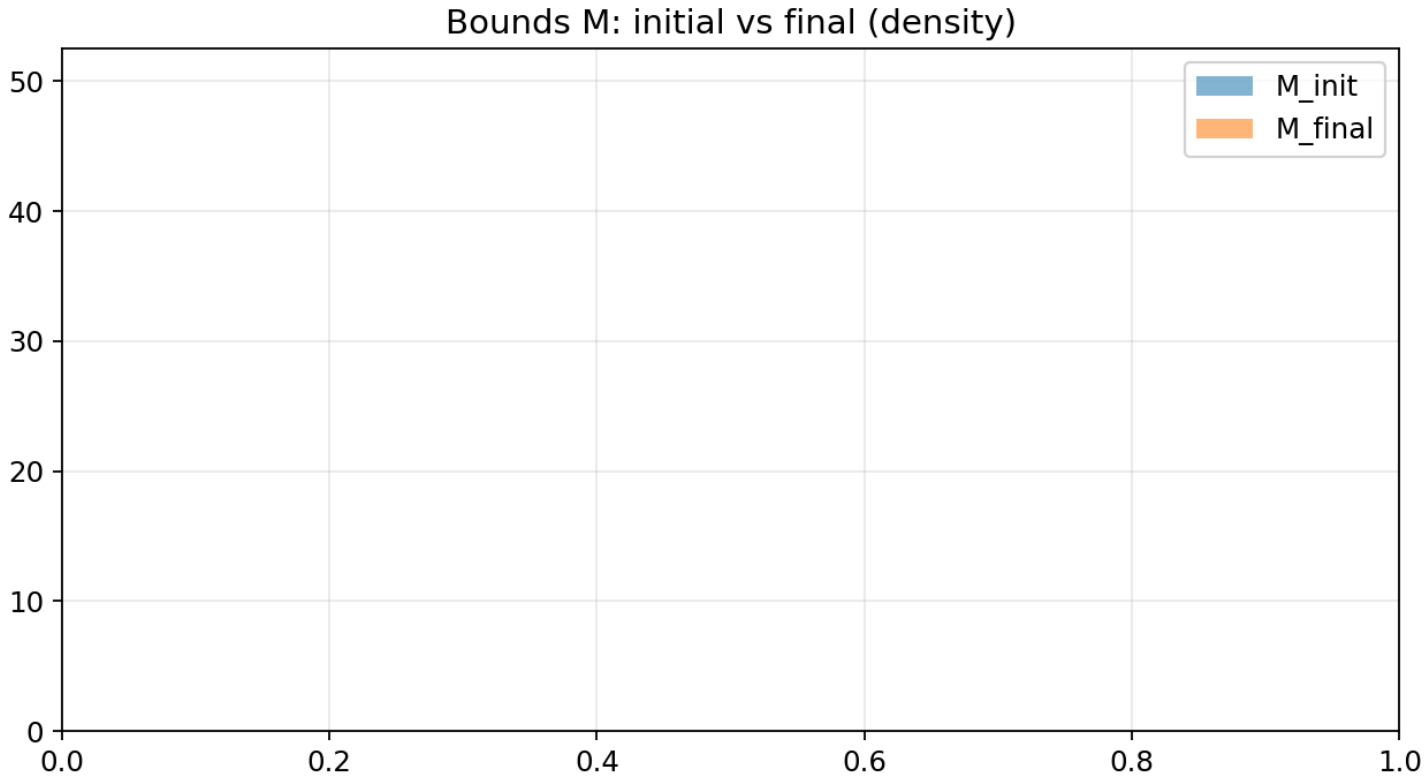
Useful to see whether the fit is uniformly good or dominated by outliers.

6.3 Learned bounds distributions

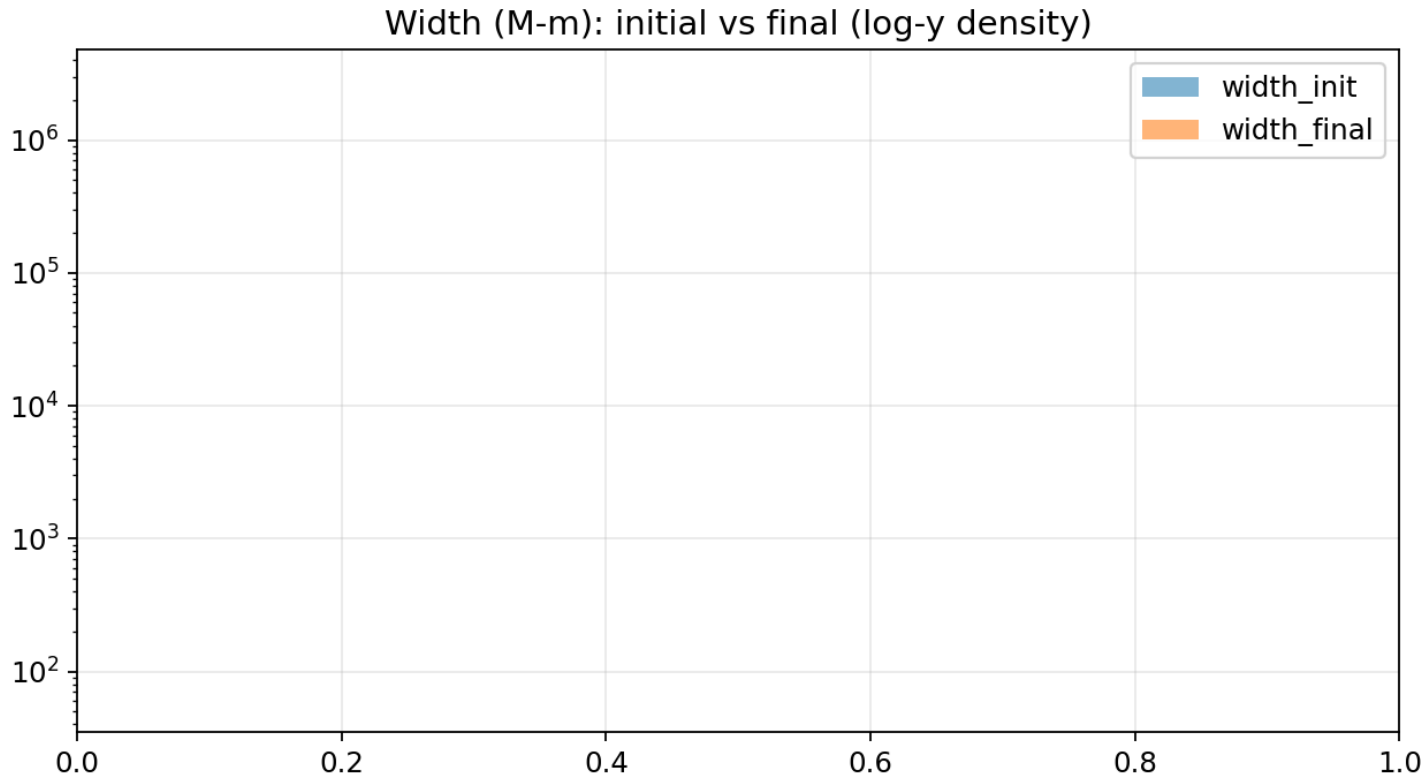
Lower bounds m



Upper bounds M



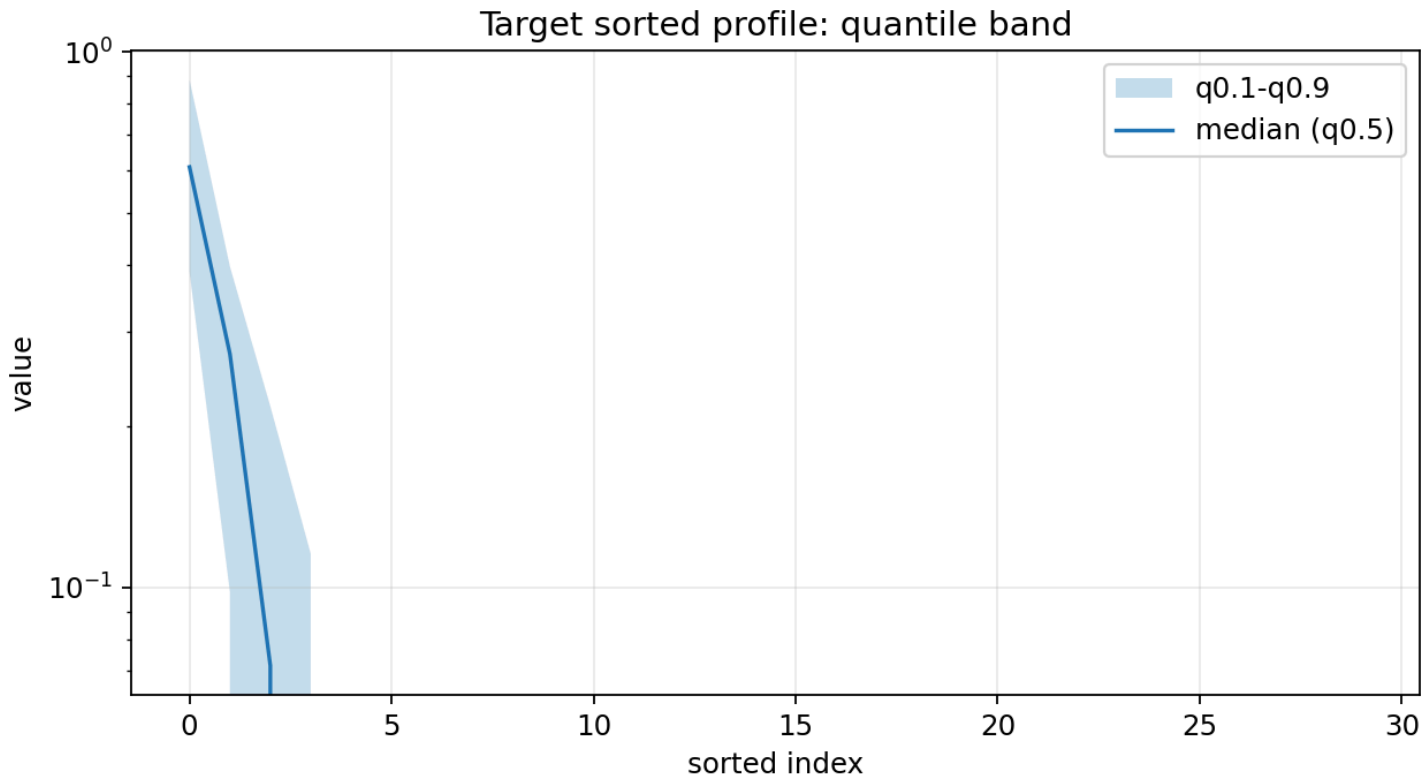
Widths $M - m$



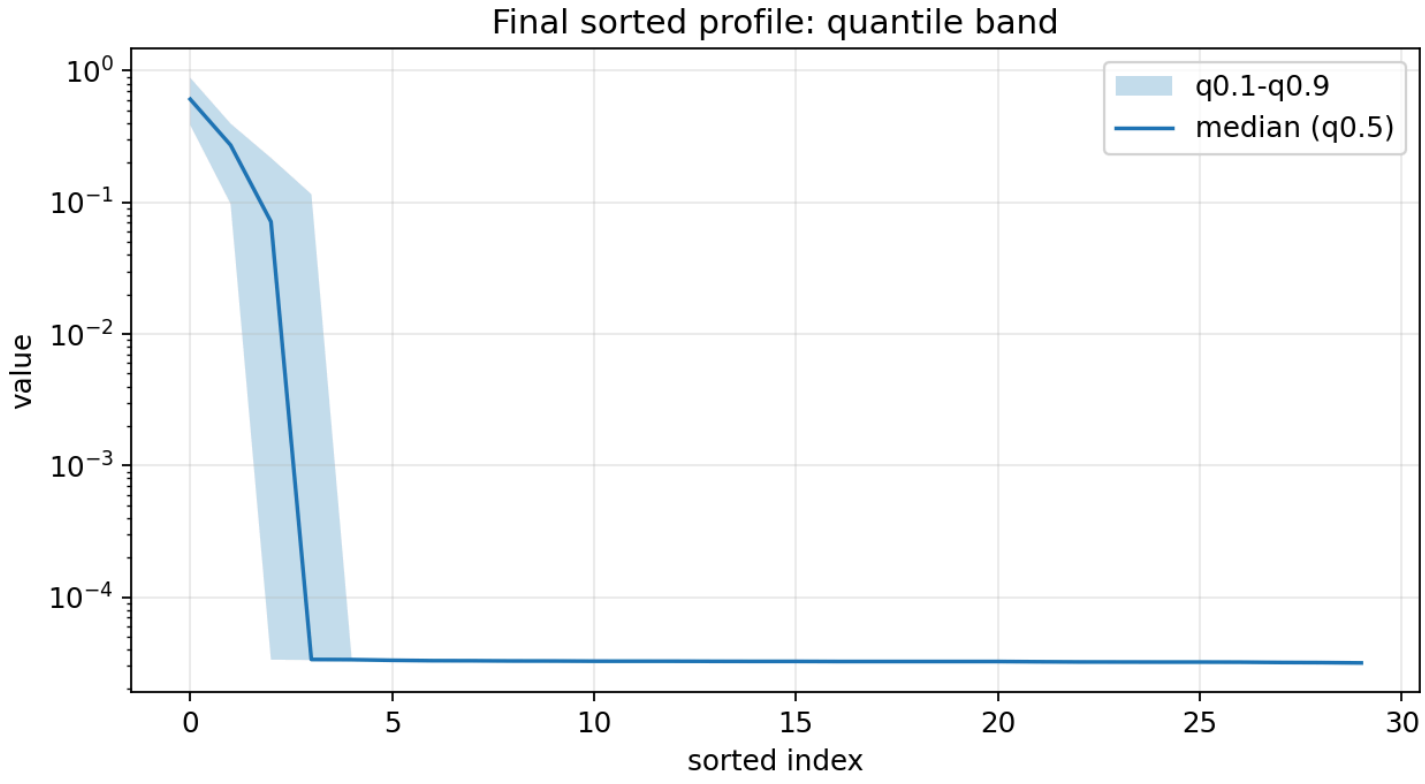
Width near 0 means a coordinate is almost fixed; width near 1 means high freedom.

6.4 Output shape match (sorted profile)

Target profile



Final learned profile



Comparing these gives an immediate intuition for what the learned feasible set is encouraging.

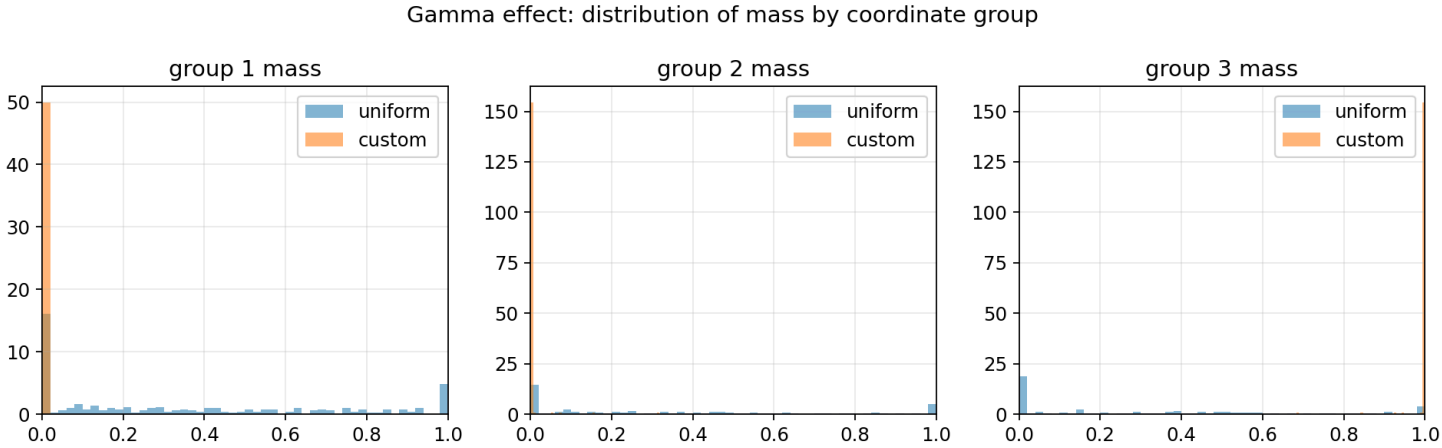
7. Use case E — Non-uniform γ (curvature / stiffness effects)

7.1 Interpretation of γ

Because the objective is $\sum_i \gamma_i (x_i - z_i)^2$:

- large γ_i makes coordinate i "stiff" (more expensive to move),
- small γ_i makes it "flexible" (cheaper to move).

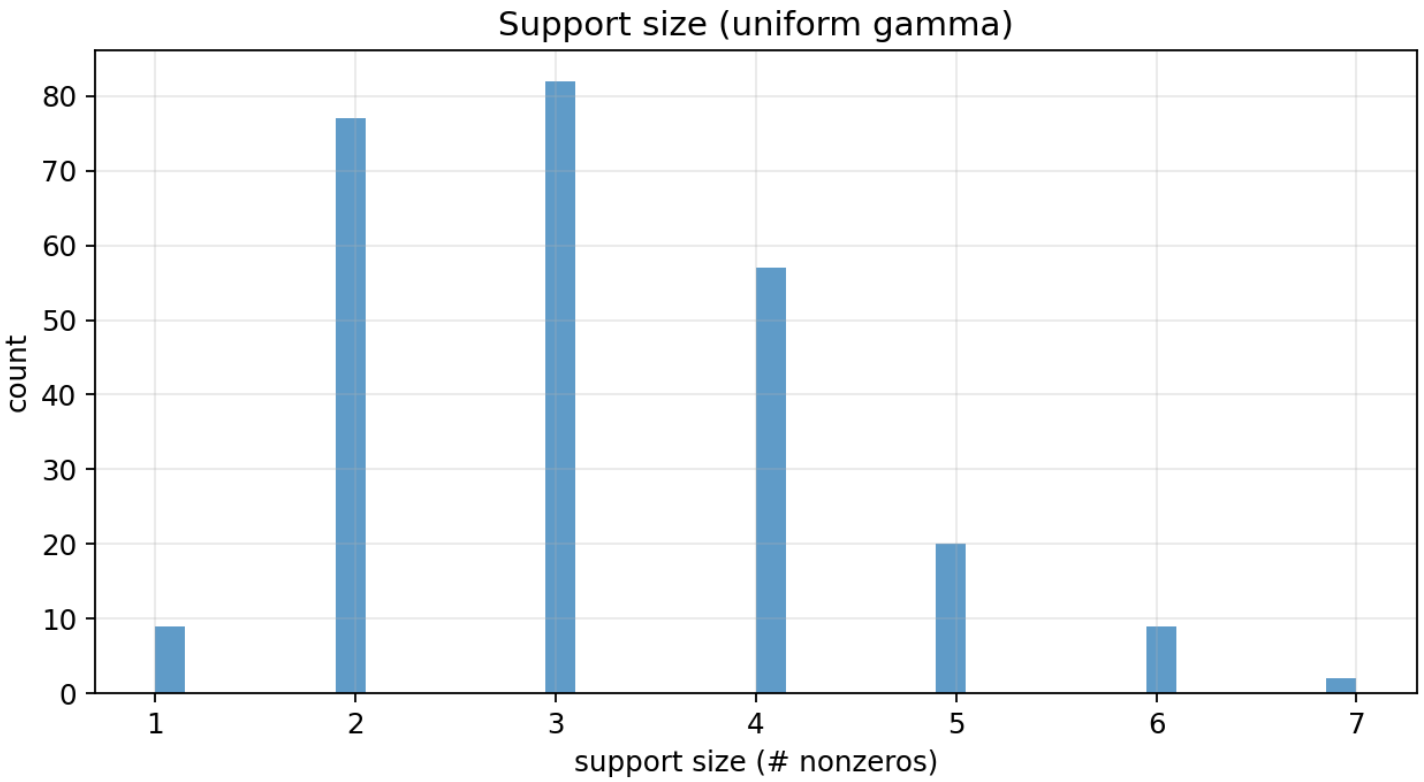
7.2 Mass allocation by coordinate group



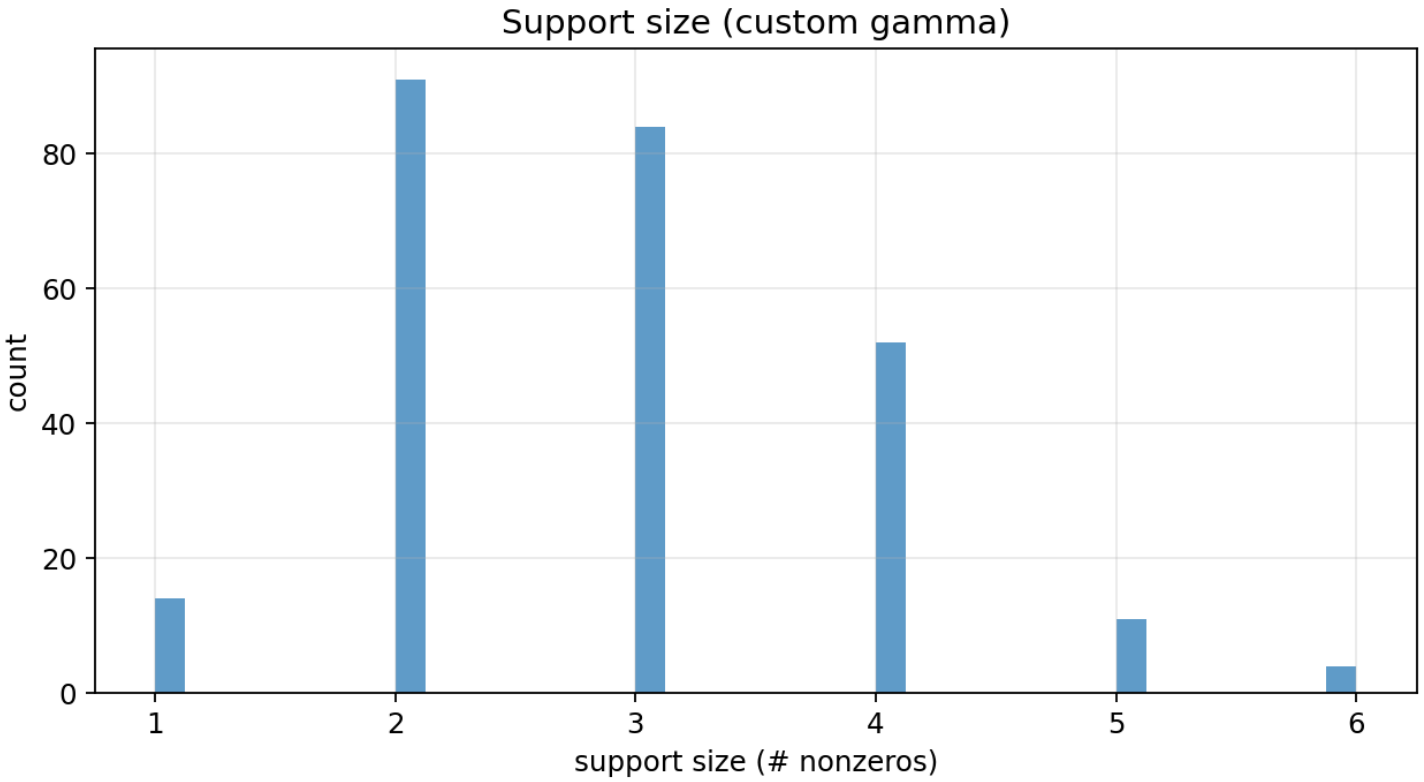
In the demo, coordinates are grouped and assigned different γ values.
You should observe systematic shifts in how much total mass each group receives.

7.3 Sparsity changes

Support size (uniform gamma)



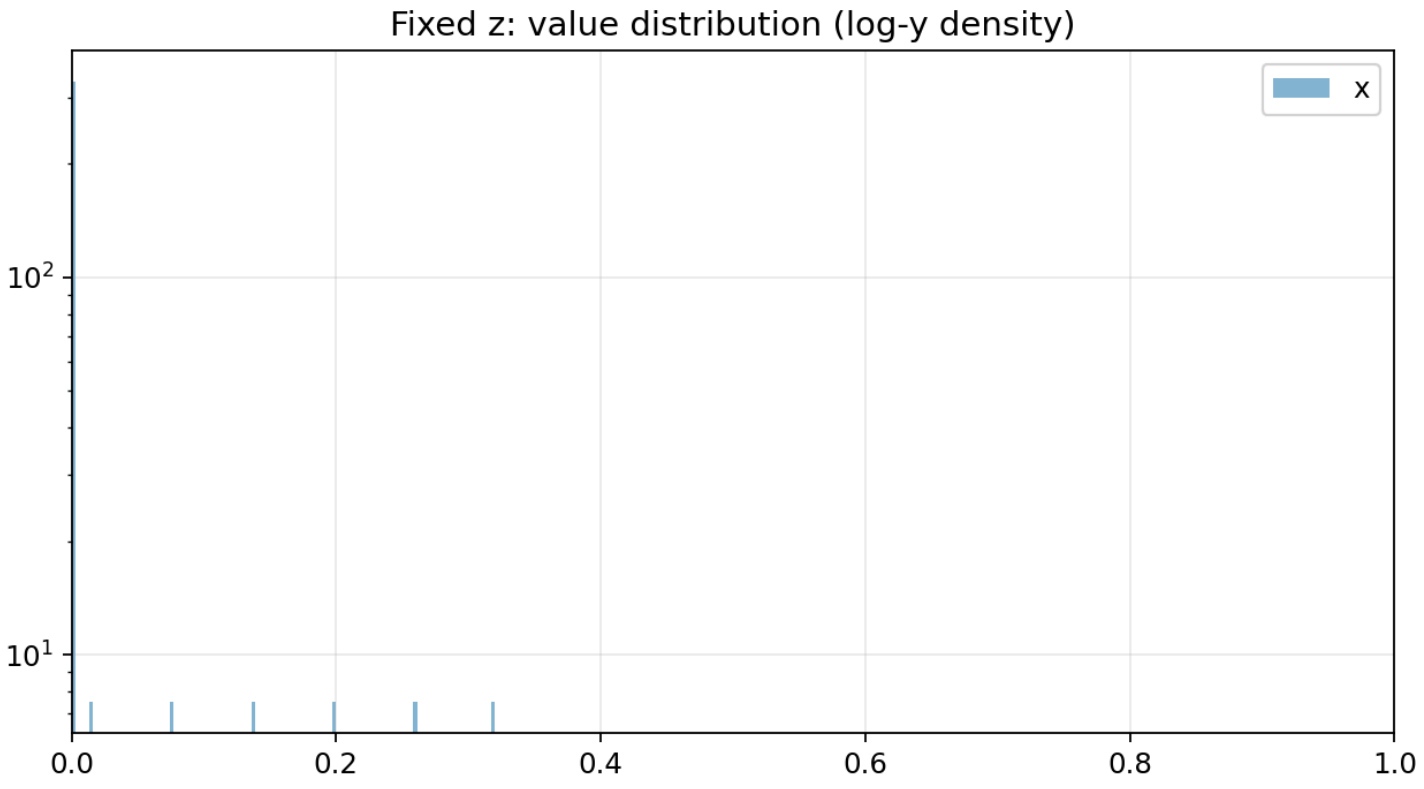
Support size (custom gamma)



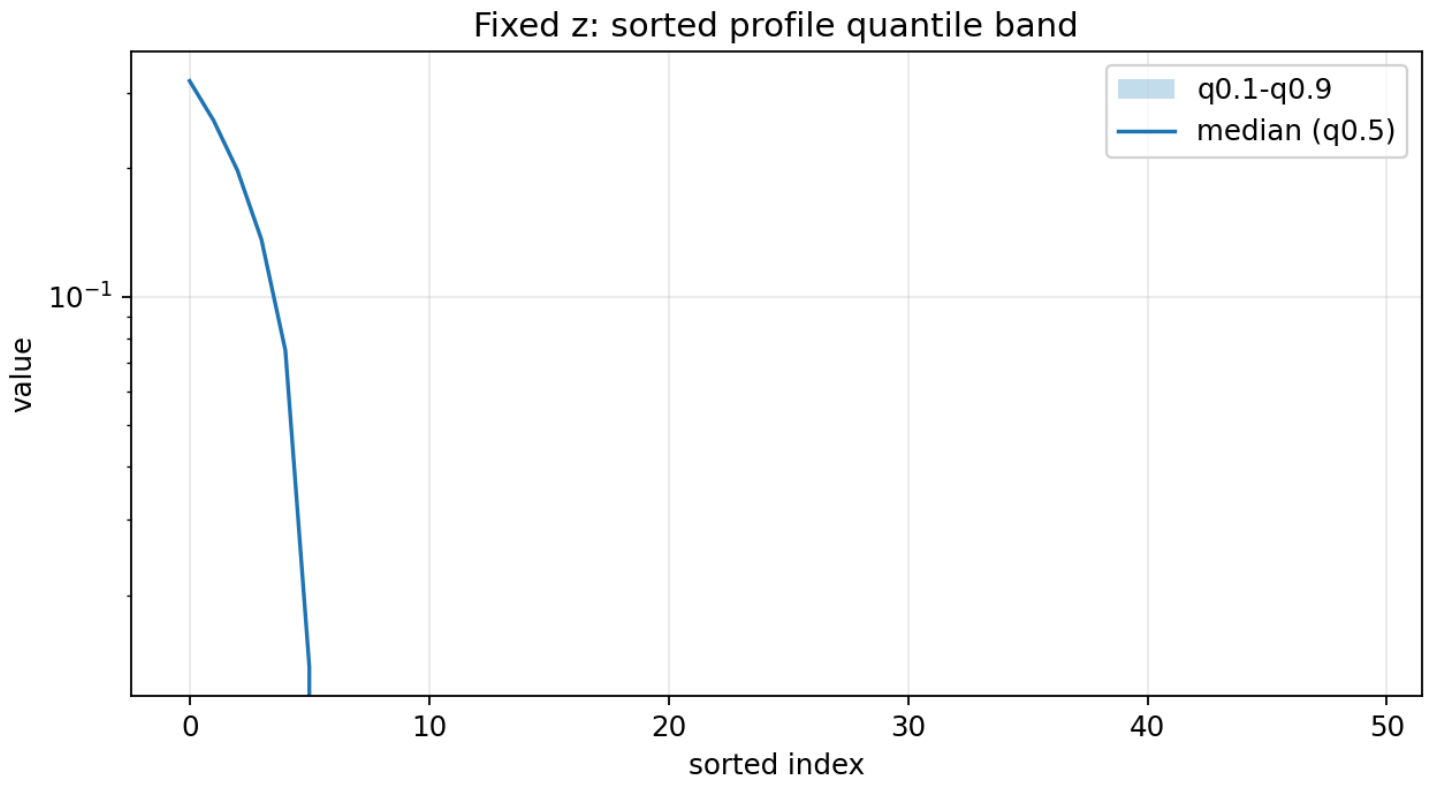
8. Use case F — Fixed z (deterministic constrained allocation)

If z is fixed, all rows produce the same x^* (up to numerical noise), so histograms show sharp spikes.

Value distribution (log-y)



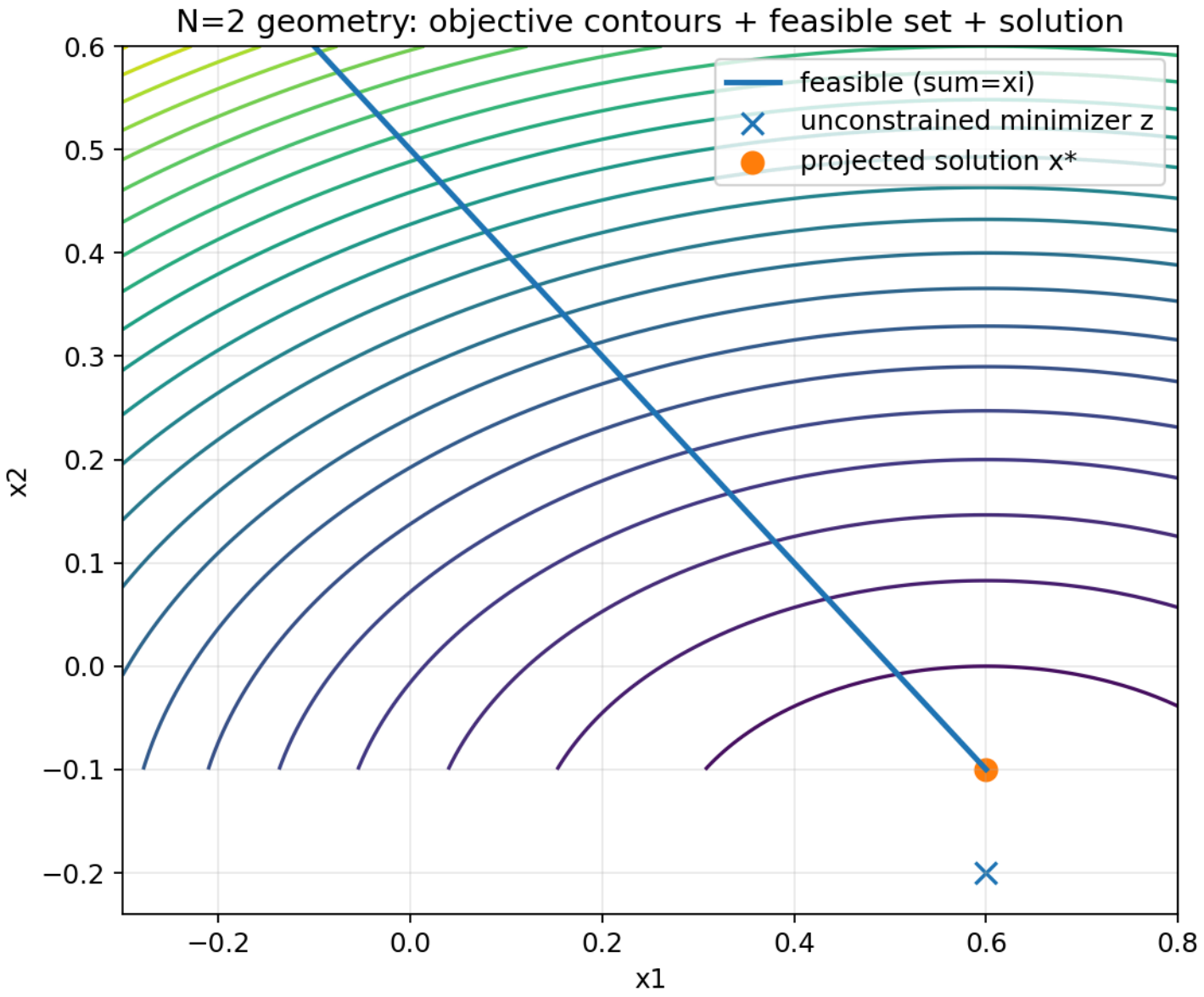
Sorted profile quantile band



9. Geometry intuition (N=2 contour plot)

This plot shows:

- objective contours of $\gamma_1(x_1 - z_1)^2 + \gamma_2(x_2 - z_2)^2$,
- the feasible segment given by $x_1 + x_2 = \xi$ inside the box,
- the solution x^*.



10. Quick "what should I expect?" checklist

If the implementation is behaving well, you typically see:

- **Feasibility:** row sums extremely close to ξ and all entries within $[m, M]$.
- **Simplex vs softmax:** projection is much sparser (support size small) than softmax.
- **k-hot:** mean row sum matches k, support size usually larger than simplex mode.
- **Learned bounds:** widths $M - m$ move away from initialization and correlate with output shape changes.
- **Gamma effects:** group mass distributions shift predictably with stiffness.

References

- Bayón, Grau, Ruiz & Suárez (2010), separable convex quadratic programming with equality and bound constraints.